

Univerzitet u Nišu
Prirodno-matematički fakultet
Departman za računarske nauke



Superrezolucija video-zapisa u realnom vremenu primenom konvolucionih neuronskih mreža

- Master rad -

Mentor
Prof. dr Branimir Todorović

Student
Igor Jovanović

Niš, 2026. godina

Sadržaj

1 Uvod **3**

2 Interpolacijske metode **4**
2.1 Interpolacija najbližeg suseda 4
2.2 Bilinearna interpolacija 4
2.3 Bikubna interpolacija 5
2.4 Lanczosova interpolacija 5

3 Arhitekture konvolucionih mreža za SISR **5**
3.1 SRCNN 6
3.1.1 Treniranje 7
3.2 VDSR - Very Deep Convolutional Networks 7
3.2.1 Treniranje 8
3.3 EDSR - Enhanced Deep Super-Resolution Network 8
3.3.1 Treniranje 10
3.3.2 Višeskalna varijanta: MDSR 10
3.4 IMDN - Information Multi-Distillation Network 10
3.5 RFDN - Residual Feature Distillation Network 11
3.6 ABPN - Anchor-Based Plain Net 12
3.7 Modifikovana arhitektura: FastEDSR 13

4 Postupak treniranja modela **14**
4.1 Skup podataka 14
4.2 Parametri treniranja 14

5 Metrike kvaliteta **15**
5.1 PSNR - Peak Signal-to-Noise Ratio 15
5.2 SSIM - Structural Similarity Index 15
5.3 LPIPS - Learned Perceptual Image Patch Similarity 15
5.4 Način računanja metrika 16

6 Merenje brzine i uslovi za rad u realnom vremenu **16**
6.1 Postupak merenja brzine 17
6.2 Izbor preciznosti 17

7 Poređenje arhitektura **18**
7.1 Kvalitet u odnosu na interpolaciju 18
7.2 Odnos kvaliteta i veličine modela 19
7.3 Odnos veličine i brzine modela 21
7.4 Odnos kvaliteta i brzine modela 21

8 Višeskalni modeli **23**

9 Superrezolucija JPEG-kompresovanih slika **24**

10 Izvršna okruženja za inferenciju **26**
10.1 ONNX Runtime . . . 27
10.2 TensorRT . . . 28
10.3 NCNN i Vulkan . . . 29
10.4 Poređenje izvršnih okruženja . . . 29

11 Implementacija demonstracionog video-plejera **31**
11.1 Mesto superrezolucije u procesu reprodukcije . . . 32
11.2 Osnovna varijanta: dekodiranje i prikaz na CPU-u . . . 32
11.2.1 Učitavanje frejmova . . . 33
11.2.2 Reprodukcija zvuka . . . 33
11.2.3 Glavna petlja . . . 34
11.2.4 Izvršna okruženja osnovne varijante . . . 34
11.3 Predobrada i postobrada . . . 35
11.4 Zero-copy varijanta: dekodiranje i prikaz na GPU-u . . . 36
11.4.1 Hardversko dekodiranje . . . 37
11.4.2 Izvršavanje modela . . . 38
11.4.3 Završno skaliranje i prikaz pomoću OpenGL-a . . . 38
11.4.4 Poređenje sa osnovnom varijantom . . . 40

12 Superrezolucija pomoću OpenGL šejdera **41**
12.1 Korisnički šejderi plejera mpv . . . 41
12.2 Prevođenje konvolucija u šejdere . . . 42
12.3 Završni prolaz: PixelShuffle i bilinearna osnova . . . 45
12.4 Ograničenja . . . 46

13 Zaključak **47**

1 Uvod

U kompjuterskoj grafici, skaliranje slike (engl. *image scaling*) označava promenu njenih prostornih dimenzija, odnosno povećanje ili smanjenje broja piksela. Kada se slika ili video-zapis prikazuje na ekranu čija je rezolucija veća od rezolucije sadržaja, sadržaj je potrebno uvećati (engl. *upscaling*). Kvalitet tog prikaza u velikoj meri zavisi od primenjene metode uvećanja. Ovaj problem je danas sve izraženiji, ekrani visokih rezolucija postali su standard, dok je velik deo postojećeg video-sadržaja dostupan u nižoj rezoluciji (stari snimci, striming pri ograničenom protoku).

Problem povećanja rezolucije pojedinačne slike (engl. *Single Image Super-Resolution*, SISR) može se formalno definisati na sledeći način: data je slika niske rezolucije $I^{LR} \in \mathbb{R}^{H \times W \times C}$ (engl. *Low Resolution*), a cilj je rekonstruisati odgovarajuću sliku visoke rezolucije $I^{HR} \in \mathbb{R}^{sH \times sW \times C}$ (engl. *High Resolution*), gde je s faktor uvećanja, a C broj kanala boje. Klasične interpolacijske metode procenjuju vrednosti novih piksela isključivo na osnovu susednih piksela. Brze su i jednostavne, ali ne mogu da rekonstruišu detalje kojih nema u ulaznoj slici, pa je rezultat uglavnom blago zamućen. Metode zasnovane na dubokom učenju, nasuprot tome, uče mapiranje između I^{LR} i I^{HR} na velikom skupu primera, pa na osnovu naučenih obrazaca mogu bolje rekonstruisati ivice i fine teksture. Savremene efikasne arhitekture pri tome postižu visok kvalitet uz relativno mali broj parametara, što otvara pitanje njihove primene i van uvećanja pojedinačnih slika.

Centralna tema ovog rada je proširenje ovog problema na uvećanje video-zapisa u realnom vremenu. Pristup je jednostavan, uvećanje pojedinačne slike primenjuje se nezavisno nad svakim frejmom. Za razliku od uvećanja pojedinačne slike, gde vreme obrade obično nije kritično, kod video-zapisa postoji strogo vremensko ograničenje: celokupna obrada jednog frejma (dekodiranje, uvećanje i priprema za prikaz) mora se završiti pre nego što treba prikazati sledeći. Za video-zapis sa 30 frejmova u sekundi raspoloživo vreme iznosi oko 33,3 ms po frejmu, a za 60 frejmova u sekundi svega oko 16,7 ms. Ako obrada traje duže, plejer mora da odbacuje frejmove, što se vidi kao trzanje (engl. *stuttering*) i gubitak fluidnosti prikaza.

Ovo ograničenje menja kriterijum izbora modela u odnosu na klasičan SISR: nije dovoljno da model daje dobar kvalitet, već mora biti i dovoljno brz. Težište rada se zato pomera sa apsolutnog kvaliteta rekonstrukcije na odnos kvaliteta i brzine. Rad nastoji da odgovori na sledeća pitanja:

- Kako se kvalitet i brzina standardnih arhitektura menjaju sa njihovom veličinom?
- Koje osobine arhitekture određuju brzinu pri zadatom kvalitetu, i da li se arhitekture efikasne po broju parametara ponašaju efikasno i po vremenu izvršavanja?
- Koliki se kvalitet može postići unutar vremenskog budžeta prikaza, i koliki deo tog budžeta odlazi na korake van same inferencije?
- U kojoj meri izvršno okruženje (PyTorch, ONNX Runtime, TensorRT, NCNN) i numerička preciznost utiču na brzinu inferencije?

Rad je organizovan na sledeći način. U poglavlju 2 opisane su klasične interpolacijske metode, a u poglavlju 3 arhitekture konvolucionih mreža za SISR (SRCNN, VDSR, EDSR, IMDN, RFDN i ABPN), kao i FastEDSR, sopstvena modifikacija EDSR-a prilagođena brzoj inferenciji. Poglavlja 4-6 opisuju postupak treniranja, metrike kvaliteta, način merenja brzine i izbor numeričke preciznosti. U poglavlju 7 data je uporedna

analiza kvaliteta, veličine i brzine svih razmatranih arhitektura, a zatim se analiziraju višeskalne varijante (poglavlje 8) i modeli trenirani za JPEG-kompresovan sadržaj (poglavlje 9). Poglavlje 10 opisuje izvoz modela u format ONNX, izvršna okruženja za inferenciju i njihovo poređenje. Poglavlja 11 i 12 opisuju primenu modela u dva sistema za uvećanje video-zapisa u realnom vremenu. Prvi je demonstracioni video-plejer u Python-u, u osnovnoj varijanti sa dekodiranjem i prikazom na procesoru (odeljak 11.2) i u zero-copy varijanti sa hardverskim dekodiranjem i prikazom na grafičkoj kartici (odeljak 11.4). Drugi je implementacija modela u vidu GLSL šejdera za postojeći plejer `mpv`, bez zavisnosti od izvršnog okruženja za neuronske mreže. Poglavlje 13 iznosi zaključke rada. Kompletan izvorni kod projekta, uključujući skripte za treniranje i evaluaciju modela, implementacije video-plejera i generator šejdera, dostupan je u repozitorijumu na adresi `https://github.com/ig0r2/super-resolution-cnn`.

2 Interpolacijske metode

Klasične metode povećanja rezolucije zasnivaju se na interpolaciji postojećih vrednosti piksela. Razmatrane metode mogu se implementirati korišćenjem različitih interpolacionih jezgara (engl. *kernels*), tako da se dvodimenzionalna interpolacija može zapisati u obliku [1]:

$$\hat{I}(x, y) = \sum_i \sum_j I(x_i, y_j)\, h(x - x_i)\, h(y - y_j), \tag{1}$$

gde je (x, y) položaj izlaznog piksela preslikan u koordinatni sistem ulazne slike, (x_i, y_j) položaji piksela ulazne slike, a h jednodimenzionalno interpolaciono jezgro. Jezgro se primenjuje posebno po svakoj osi. Metode se razlikuju po obliku i veličini jezgra, što utiče na računsku složenost i kvalitet rekonstruisane slike.

2.1 Interpolacija najbližeg suseda

Kod interpolacije najbližeg suseda (engl. *Nearest Neighbor*) svaki novi piksel dobija vrednost prostorno najbližeg piksela ulazne slike. Ova operacija odgovara pravougaonom jezgru:

$$h(x) = \begin{cases} 1, & |x| < 0{,}5, \\ 0, & \text{inače.} \end{cases} \tag{2}$$

Ova metoda ima najmanju računsku složenost, ali pri uvećanju slike dovodi do izraženih blokova iste boje, jer se jedna vrednost ulaznog piksela preslikava na više izlaznih piksela.

2.2 Bilinearna interpolacija

Kod bilinearne (engl. *Bilinear*) interpolacije vrednost novog piksela određuje se težinskom kombinacijom četiri najbliža piksela. Odgovarajuće jednodimenzionalno jezgro je trougaonog oblika:

$$h(x) = \begin{cases} 1 - |x|, & |x| < 1, \\ 0, & \text{inače.} \end{cases} \tag{3}$$

Rezultat je vizuelno glađi nego kod interpolacije najbližeg suseda, ali interpolacija istovremeno prigušuje deo visokofrekventnih komponenti slike, što se primećuje kao zamućenje.

2.3 Bikubna interpolacija

Bikubna (engl. *Bicubic*) interpolacija koristi 4×4 okolinu, odnosno 16 susednih piksela. Kubno interpolaciono jezgro ima oblik [2]:

$$h(x) = \begin{cases} 1{,}5|x|^3 - 2{,}5|x|^2 + 1, & |x| < 1, \\ -0{,}5|x|^3 + 2{,}5|x|^2 - 4|x| + 2, & 1 \leq |x| < 2, \\ 0, & \text{inače.} \end{cases} \tag{4}$$

Bikubna interpolacija predstavlja jednu od najčešće korišćenih metoda skaliranja slike. U literaturi o superrezoluciji naročito je česta upotreba bikubnog smanjivanja rezolucije za generisanje sintetičkih LR slika iz odgovarajućih HR slika, kao i referentna metoda pri evaluaciji modela.

2.4 Lanczosova interpolacija

Lanczosova interpolacija koristi prozorsko sinc jezgro [3]:

$$h(x) = \begin{cases} 1, & x = 0, \\ \dfrac{a \sin(\pi x) \sin(\frac{\pi x}{a})}{\pi^2 x^2}, & 0 < |x| < a, \\ 0, & |x| \geq a, \end{cases} \tag{5}$$

gde parametar a, najčešće $a = 2$ ili $a = 3$, određuje veličinu jezgra. Lanczosova interpolacija može bolje očuvati oštre prelaze i visokofrekventne komponente od prethodno navedenih interpolacionih metoda, ali može doći do pojave artefakata prstenovanja (engl. *ringing*).

Zajedničko ograničenje interpolacionih metoda jeste to što izlaz određuju isključivo na osnovu lokalnih vrednosti postojećih piksela. One ne koriste prethodno naučeno znanje o strukturi prirodnih slika i zato ne mogu pouzdano rekonstruisati nedostajuće fine teksture.

3 Arhitekture konvolucionih mreža za SISR

Pristupi zasnovani na dubokom učenju najčešće formulišu problem SISR kao regresioni zadatak. Cilj je naučiti parametre mreže tako da njen izlaz bude što sličniji odgovarajućoj referentnoj HR slici. Ako sa I^{LR} označimo ulaz mreže, parametri mreže θ mogu se odrediti minimizacijom funkcije gubitka:

$$\hat{\theta} = \arg\min_{\theta} L\Big(F_\theta(I^{LR}), I^{HR}\Big), \tag{6}$$

gde F_θ predstavlja preslikavanje koje realizuje neuronska mreža, dok L označava funkciju gubitka.

Različite arhitekture razlikuju se po organizaciji slojeva, dubini i širini mreže, mestu na kome se vrši uvećanje prostorne rezolucije, upotrebi rezidualnih veza i mehanizama

pažnje, kao i po strategiji treniranja. Takođe, u zavisnosti od arhitekture, ulaz mreže može biti originalna LR slika ili LR slika koja je prethodno interpolirana na ciljnu rezoluciju.

3.1 SRCNN

SRCNN [4] predstavlja jednu od prvih uspešnih primena konvolucionih neuronskih mreža za direktno učenje mapiranja između slika niske i visoke rezolucije. Značajan doprinos rada predstavlja dokaz da se postupci zasnovani na retkom kodiranju (engl. *sparse coding*) [5] mogu interpretirati kroz niz konvolucionih operacija.

Kod SRCNN-a ulaz u mrežu nije originalna LR slika. Ona se najpre bikubnom interpolacijom uvećava na željene prostorne dimenzije, obeležićemo je sa I_{bic}^{LR}. Mreža stoga sama ne povećava broj piksela, već transformiše prethodno interpoliranu sliku:

$$I^{SR} = F_\theta(I_{bic}^{LR}). \tag{7}$$

Osnovna SRCNN arhitektura sastoji se od tri konvoluciona sloja, od kojih svaki ima konceptualno posebnu ulogu.

1. Ekstrakcija i reprezentacija isečaka (engl. *patch extraction and representation*): prvi konvolucioni sloj primenjuje n_1 filtera sa konvolucionim jezgrom (engl. *kernel*) veličine $f_1 \times f_1$ nad ulaznim kanalima, nakon čega se primenjuje ReLU aktivaciona funkcija:

$$F_1(Y) = \max(0,\, W_1 * Y + B_1). \tag{8}$$

Ovaj korak konceptualno odgovara izdvajanju isečaka i njihovom predstavljanju pomoću rečnika u pristupima zasnovanim na retkom kodiranju.

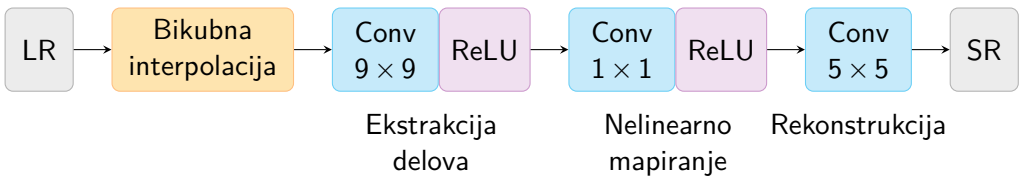
2. Nelinearno mapiranje (engl. *non-linear mapping*): drugi konvolucioni sloj koristi n_2 filtera nad n_1 ulaznih mapa obeležja (engl. *feature maps*) sa konvolucionim jezgrom veličine $f_2 \times f_2$, čime se transformiše reprezentacija dobijena prethodnim slojem:

$$F_2(Y) = \max(0,\, W_2 * F_1(Y) + B_2). \tag{9}$$

U osnovnoj konfiguraciji koristi se konvoluciono jezgro veličine 1×1. Takva konvolucija kombinuje informacije između kanala na istoj prostornoj poziciji, bez korišćenja piksela na susednim pozicijama.

3. Rekonstrukcija: poslednji konvolucioni sloj kombinuje dobijena obeležja iz susednih pozicija i formira izlaznu sliku, bez korišćenja aktivacione funkcije:

$$F(Y) = W_3 * F_2(Y) + B_3. \tag{10}$$



Slika 1: Arhitektura SRCNN mreže.

U standardnom modelu koji obrađuje samo luminansni (Y) kanal ulaz i izlaz imaju samo jedan kanal, dok proširena verzija koja istovremeno obrađuje tri kanala ima tri ulazna i tri izlazna kanala. Standardna konfiguracija koristi $f_1 = 9$, $f_2 = 1$, $f_3 = 5$, $n_1 = 64$ i $n_2 = 32$. Promenom broja filtera i veličine njihovih konvolucionih jezgara moguće je menjati odnos između kvaliteta rekonstrukcije i računske složenosti.

3.1.1 Treniranje

Osnovni SRCNN eksperimenti koriste trening skup *T91* [5], koji sadrži 91 sliku. Iz njih se izdvajaju približno 24 800 preklapajućih isečaka veličine 33×33 piksela, sa korakom (engl. *stride*) od 14 piksela. U osnovnim eksperimentima sa ovim skupom ne koristi se geometrijsko proširivanje skupa podataka (engl. *data augmentation*), poput rotiranja i refleksije slika. Većina eksperimenata sprovodi se nad kanalom luminanse, slike se konvertuju u YCbCr prostor boja, a model obrađuje samo Y kanal. U proširenoj verziji rada autori eksperimentišu i sa mrežom koja istovremeno obrađuje sva tri kanala, uključujući direktno treniranje u RGB prostoru. Kao funkcija gubitka koristi se srednja kvadratna greška (engl. *Mean Squared Error*, MSE), odnosno ℓ_2 gubitak:

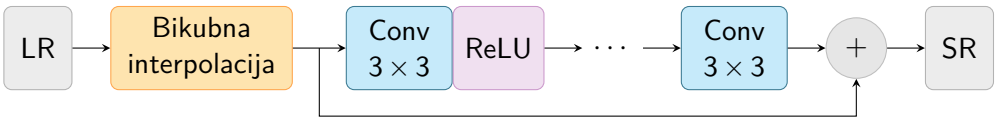
$$L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left\| I_i^{SR} - I_i^{HR} \right\|_2^2. \tag{11}$$

Optimizacija se sprovodi stohastičkim gradijentnim spustom (engl. *Stochastic Gradient Descent*, SGD). U osnovnoj konfiguraciji autori koriste stopu učenja 10^{-4} za prva dva sloja i manju stopu 10^{-5} za poslednji sloj. Model se trenira kroz 8×10^8 propagacija greške unazad.

3.2 VDSR - Very Deep Convolutional Networks

VDSR [6] značajno povećava dubinu konvolucionih modela za SISR i zasnovan je na principima VGG mreže [7]. Za razliku od troslojnog SRCNN-a, VDSR koristi 20 konvolucionih slojeva. Ključne ideje koje omogućavaju uspešno treniranje ovako duboke mreže jesu rezidualno učenje, visoka početna stopa učenja i adaptivno odsecanje gradijenata.

Prvi konvolucioni sloj generiše 64 mape obeležja pomoću konvolucionih jezgara veličine 3×3. Nakon njega sledi 18 konvolucionih slojeva sa po 64 izlazna kanala i jezgrima veličine 3×3, dok poslednji, dvadeseti sloj generiše rezidualnu sliku. ReLU aktivaciona funkcija primenjuje se nakon prvih 19 konvolucionih slojeva, dok poslednji sloj nema aktivacionu funkciju. Zbog sukcesivne primene 20 konvolucija sa jezgrima veličine 3×3, receptivno polje izlaznog piksela iznosi 41×41 piksela. Na taj način mreža može da koristi znatno širi prostorni kontekst nego SRCNN bez potrebe za korišćenjem velikih konvolucionih jezgara.



Slika 2: Arhitektura VDSR mreže.

Kao i SRCNN, VDSR na ulazu prima LR sliku koja je prethodno uvećana bikubnom interpolacijom na ciljnu rezoluciju. Međutim, umesto direktnog predviđanja HR slike, mreža uči rezidual:

$$r = I^{HR} - I_{bic}^{LR} \tag{12}$$

$$\hat{\theta} = \arg\min_{\theta} L\Big(F_\theta(I_{bic}^{LR}), r\Big). \tag{13}$$

Izlazna SR slika dobija se sabiranjem predviđenog reziduala i interpolirane ulazne slike:

$$I^{SR} = I^{LR}_{bic} + F_\theta(I^{LR}_{bic}). \tag{14}$$

Intuicija ovog pristupa jeste da bikubno interpolirana slika već sadrži veliki deo niskofrekventne strukture originalne slike. Mreža zato ne mora da uči modelovanje cele slike, već samo kako da poboljša već interpoliranu sliku. Autori pokazuju da ovakvo rezidualno učenje znatno ubrzava konvergenciju.

3.2.1 Treniranje

VDSR takođe koristi srednju kvadratnu grešku kao funkciju gubitka. Optimizacija se sprovodi stohastičkim gradijentnim spustom sa momentom 0,9 i regularizacijom težina (engl. *weight decay*) od 10^{-4}. Posebno važan deo postupka treniranja predstavlja veoma visoka početna stopa učenja. Ona je postavljena na 0,1, a smanjuje se deset puta svakih 20 epoha, tokom ukupno 80 epoha treniranja. Visoka stopa učenja ubrzava konvergenciju, ali istovremeno povećava rizik od pojave veoma velikih gradijenata, zbog čega VDSR uvodi adaptivno odsecanje gradijenata (engl. *adjustable gradient clipping*). Gradijenti se ograničavaju na interval $\left[-\frac{\delta}{\gamma}, \frac{\delta}{\gamma}\right]$, gde je γ trenutna stopa učenja, dok je δ parametar odsecanja. Na ovaj način se efektivni prag odsecanja prilagođava promeni stope učenja. Kombinacija ove tehnike i rezidualnog učenja omogućava stabilno korišćenje visoke početne stope učenja.

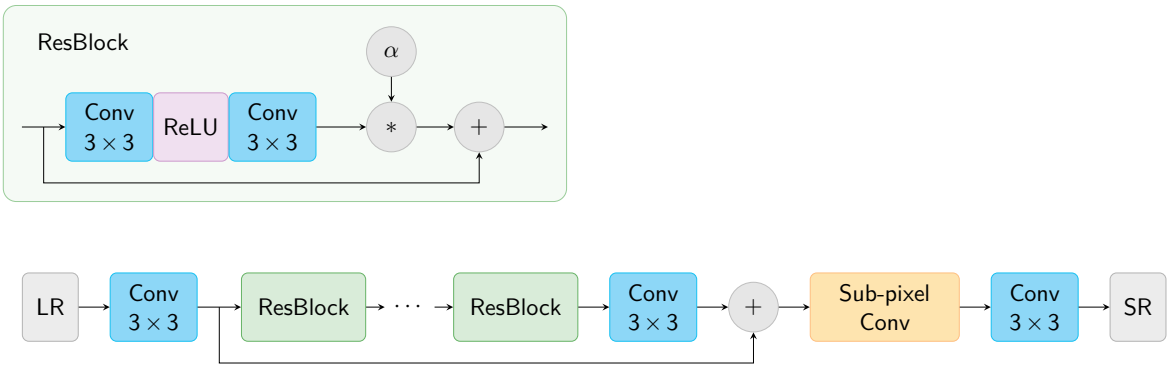
Važan doprinos VDSR-a predstavlja i višeskalno treniranje (engl. *multi-scale training*). Umesto zasebnog modela za svaki faktor uvećanja ($\times 2$, $\times 3$ i $\times 4$), isti model može se trenirati primerima iz više skala. Pošto je ulaz mreže u svim slučajevima već interpoliran na ciljnu rezoluciju, arhitekturu nije potrebno menjati u zavisnosti od faktora uvećanja. Autori pokazuju da ovakav višeskalni model postiže rezultate uporedive sa odgovarajućim modelima treniranim za pojedinačne faktore uvećanja.

3.3 EDSR - Enhanced Deep Super-Resolution Network

EDSR [8] je razvijen polazeći od SRResNet [9] arhitekture, pri čemu autori pojednostavljuju rezidualne blokove i zatim značajno povećavaju širinu i dubinu mreže.

Važna razlika u odnosu na SRCNN i VDSR jeste mesto na kome se vrši uvećanje rezolucije. EDSR ne prima unapred bikubno uvećanu sliku, već direktno obrađuje LR sliku u njenoj originalnoj prostornoj rezoluciji. Veći deo izračunavanja zato se obavlja u LR prostoru, a uvećanje se vrši tek pri kraju mreže.

Mreža počinje konvolucionim slojem koji izdvaja početna obeležja iz LR slike. Sledi telo mreže, koje se sastoji od niza rezidualnih blokova [10] i dodatne konvolucije. Izlaz tela se sabira sa početnim obeležjima preko duge rezidualne veze. Prostorna rezolucija se uvećava tek nakon toga, pomoću sub-pikselne konvolucije (engl. *sub-pixel convolution*) [11]. Na kraju završni konvolucioni sloj od uvećanih obeležja formira SR sliku.

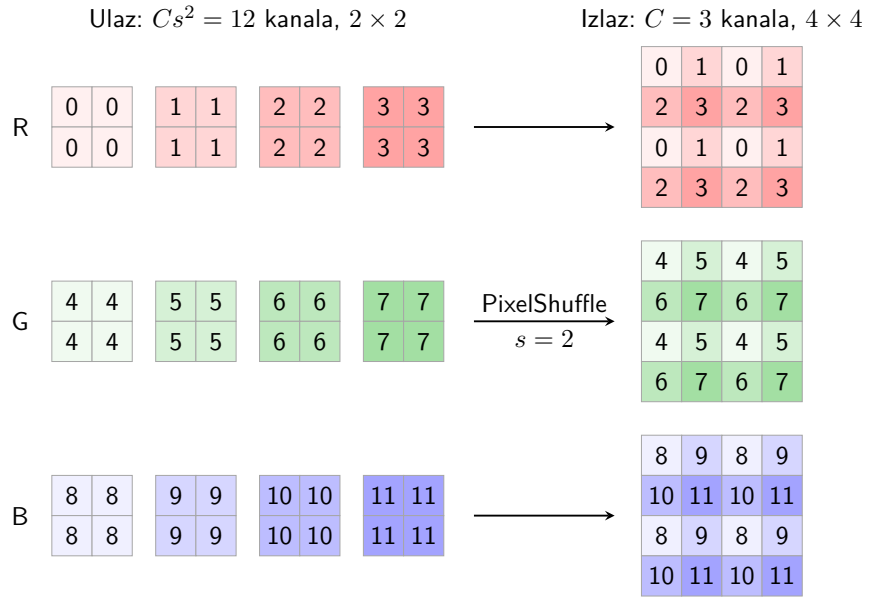


Slika 3: Arhitektura EDSR mreže.

Sub-pikselna konvolucija sastoji se od konvolucije praćene operacijom preraspoređivanja piksela (engl. *Pixel Shuffle*). Operacija PixelShuffle nema parametre i ne vrši nikakvo izračunavanje, već samo preraspoređuje elemente tenzora. Za faktor uvećanja s, prethodna konvolucija daje tenzor sa Cs^2 kanala dimenzija $H \times W$, a PixelShuffle ga preslikava u tenzor sa C kanala dimenzija $sH \times sW$:

$$\mathbf{Y}_{c,\,y,\,x} = \mathbf{X}_{cs^2 + s\,(y \bmod s) + (x \bmod s),\ \lfloor y/s \rfloor,\ \lfloor x/s \rfloor}. \tag{15}$$

Drugim rečima, s^2 kanala jednog LR piksela raspoređuju se u blok $s \times s$ susednih piksela izlazne slike. Konvolucija ispred PixelShuffle tako uči da za svaki LR piksel predvidi s^2 odgovarajućih HR piksela, a celo uvećanje se izračunava u LR prostoru. Primer za $s = 2$ i $C = 3$ prikazan je na slici 4.



Slika 4: PixelShuffle za $s = 2$ i $C = 3$. Brojevi označavaju indeks ulaznog kanala iz kog potiče vrednost, a svaki blok 2×2 na izlazu potiče od istog LR piksela.

Jedna od najvažnijih izmena u odnosu na SRResNet jeste potpuno uklanjanje slojeva za batch normalizaciju (engl. *batch normalization*) [12] iz rezidualnih blokova. Autori navode da normalizacija ograničava opseg vrednosti mapa obeležja, što nije poželjno kod zadataka rekonstrukcije slike. Eksperimentalno je pokazano da uklanjanje batch normalizacije poboljšava kvalitet rekonstrukcije. Dodatna prednost je značajno smanjenje memorijske potrošnje tokom treniranja.

Osnovna EDSR varijanta koristi 16 rezidualnih blokova i 64 kanala. U punom EDSR modelu broj blokova povećan je na $B = 32$, dok svaki konvolucioni sloj koristi $n_f = 256$ kanala. Ovakav model ima približno 43 miliona parametara. Pri ovako velikom broju mapa obeležja autori uočavaju numeričku nestabilnost tokom treniranja. Zbog toga uvode konstantno skaliranje izlaza rezidualne grane faktorom α pre sabiranja sa ulazom bloka:

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \alpha\mathcal{F}(\mathbf{x}_l), \tag{16}$$

gde $\mathcal{F}(\mathbf{x}_l)$ označava transformaciju koju realizuje rezidualna grana. Ovakvo skaliranje značajno stabilizuje treniranje veoma širokog modela. U osnovnoj EDSR varijanti sa 64 kanala skaliranje reziduala nije neophodno, dok se u punom modelu koristi faktor $\alpha = 0{,}1$.

3.3.1 Treniranje

EDSR se trenira na DIV2K [13] skupu podataka. Kao funkcija gubitka koristi se ℓ_1 norma:

$$L(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left\|I_i^{SR} - I_i^{HR}\right\|_1. \tag{17}$$

Autori navode da je u njihovim eksperimentima ℓ_1 gubitak omogućio bolju konvergenciju i nešto bolje rezultate od ℓ_2 gubitka. Optimizacija se sprovodi Adam algoritmom. Za treniranje se koriste LR isečci veličine 48×48, a početna stopa učenja postavljena je na 10^{-4} i prepolovljava se nakon svakih 2×10^5 ažuriranja parametara.

Karakterističan detalj jeste inicijalizacija modela za veće faktore uvećanja. Pri treniranju $\times 3$ i $\times 4$ modela koriste se parametri prethodno istreniranog $\times 2$ modela tamo gde su arhitekture kompatibilne. Autori pokazuju da ovakva inicijalizacija prethodno naučenim parametrima ubrzava konvergenciju i poboljšava konačne rezultate u odnosu na treniranje modela za veće faktore potpuno od početka.

3.3.2 Višeskalna varijanta: MDSR

Osnovni EDSR predstavlja model za pojedinačni faktor uvećanja, odnosno zaseban model se trenira za svaki faktor. Autori zato predlažu i višeskalnu varijantu MDSR, koja omogućava rekonstrukciju za faktore $\times 2$, $\times 3$ i $\times 4$ pomoću jedne mreže.

Veći deo mreže je zajednički za sve faktore uvećanja. Na početku se koriste posebni pretprocesni moduli za svaki faktor. Svaki od njih sadrži dva rezidualna bloka sa konvolucionim jezgrima veličine 5×5, čime se u ranoj fazi mreže dobija veće receptivno polje uz relativno plitak deo mreže specifičan za dati faktor uvećanja. Nakon zajedničkog tela mreže koriste se posebni moduli za uvećanje za svaki faktor. Na taj način najveći deo parametara deli se između različitih faktora uvećanja, što je efikasnije od održavanja više potpuno nezavisnih modela.

3.4 IMDN - Information Multi-Distillation Network

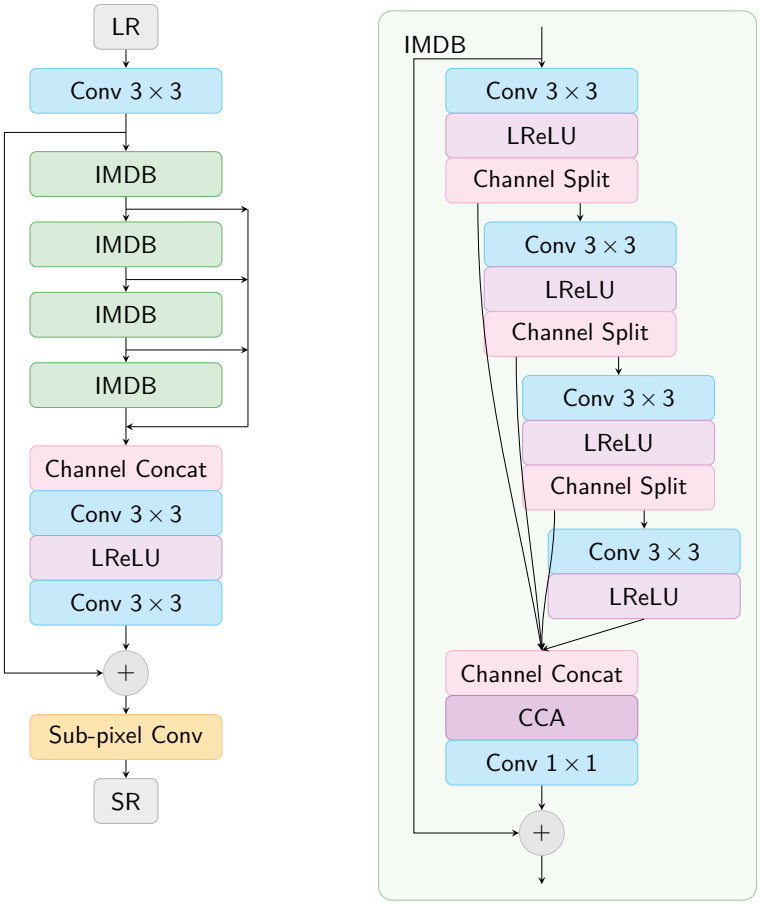
IMDN [14] pripada grupi efikasnijih SISR arhitektura, čiji je cilj smanjenje računske složenosti i broja parametara uz što manji gubitak kvaliteta rekonstruisane slike.

Ključna ideja IMDN-a je destilacija obeležja (engl. *feature distillation*), pri kojoj se deo kanala postepeno izdvaja tokom obrade, dok se preostali kanali prosleđuju narednim slojevima. Mreža je sastavljena od blokova za višestruku destilaciju informacija

(engl. *Information Multi-Distillation Block*, IMDB). Osnovu IMDB-a čini postepeno izdvajanje dela kanala tokom obrade. Posle svake konvolucije izlaz se deli na dve grupe: destilovana obeležja, koja se čuvaju za kasnije objedinjavanje, i preostala obeležja, koja se prosleđuju sledećoj konvoluciji. Na kraju bloka se ta destilovana obeležja spajaju. Nakon toga sledi mehanizam pažnje po kanalima zasnovan na kontrastu (engl. *Contrast-Aware Channel Attention*, CCA), varijanta pažnje po kanalima uvedene u radu [15]. Za svaki kanal CCA koristi njegovu srednju vrednost i standardnu devijaciju, čime se u procenu značaja kanala uključuje i informacija o prostornom kontrastu. Posle CCA se primenjuje konvolucija sa jezgrom veličine 1×1, kojom se broj kanala svodi na dimenziju ulaza bloka i taj izlaz se sabira sa ulazom bloka preko lokalne rezidualne veze.

Za razliku od prethodno opisanih arhitektura, koje koriste ReLU aktivacionu funkciju, IMDN posle konvolucija primenjuje *Leaky ReLU*, $\mathrm{LReLU}(x) = \max(x, \alpha x)$, koja za negativne ulaze ne vraća nulu, već vrednost umanjenu faktorom α. Podrazumevana vrednost je $\alpha = 0{,}05$.

Prostorna rezolucija se, kao i kod EDSR modela, uvećava sub-pikselnom konvolucijom na kraju mreže. Za razliku od EDSR-a, posle nje ne sledi završni konvolucioni sloj, pa sub-pikselna konvolucija direktno daje SR sliku.



Slika 5: Arhitektura IMDN mreže.

3.5 RFDN - Residual Feature Distillation Network

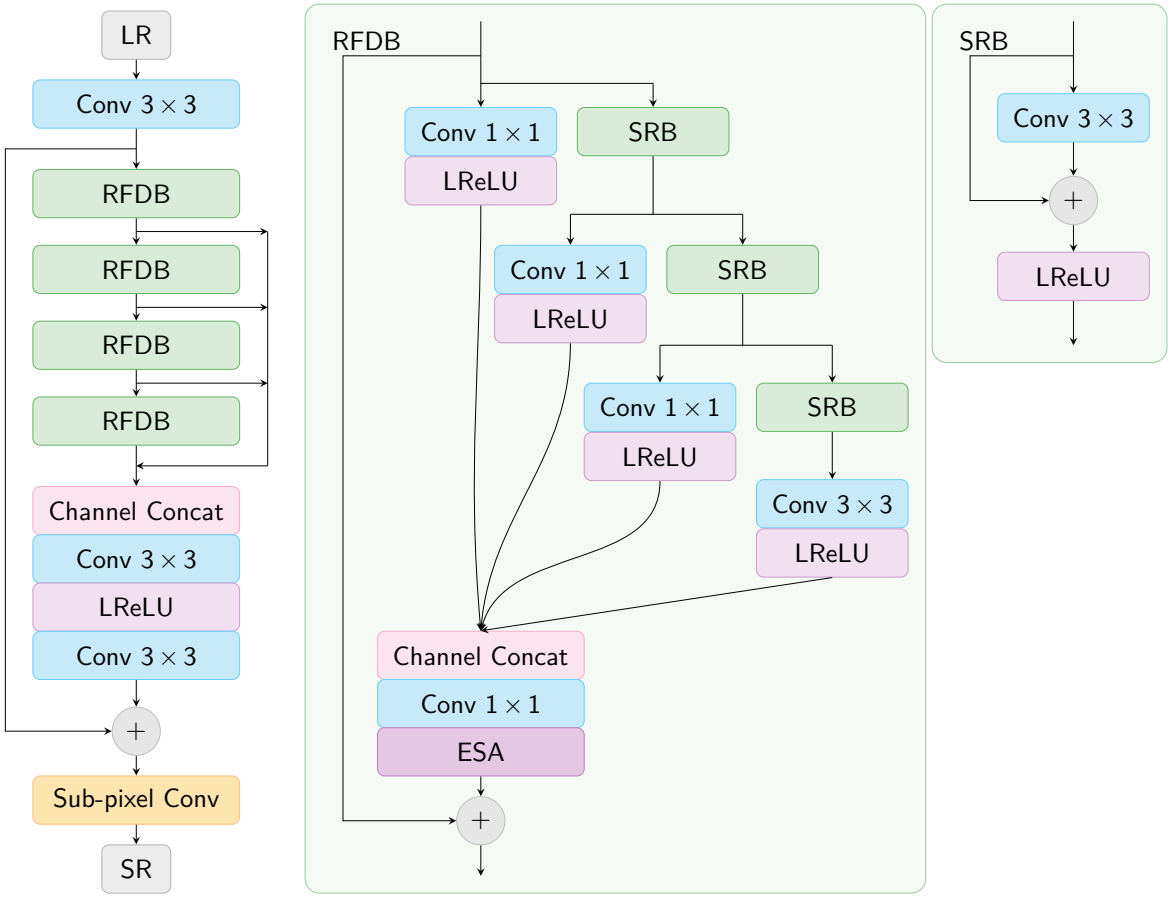
RFDN [16] zadržava princip postepene destilacije obeležja, ali menja način na koji se destilovani kanali izdvajaju. Mreža je sastavljena od blokova za rezidualnu destilaci-

ju obeležja (engl. *Residual Feature Distillation Block*, RFDB). Umesto deljenja izlaza konvolucije po kanalima, RFDB koristi vezu za destilaciju obeležja (engl. *Feature Distillation Connection*, FDC), u kojoj isti ulaz obrađuju dve odvojene grane. Destilaciona grana koristi 1×1 konvoluciju za direktno izdvajanje manjeg broja kanala, dok druga grana nastavlja obradu pomoću plitkog rezidualnog bloka (engl. *Shallow Residual Block*, SRB), odnosno 3×3 konvolucije sa rezidualnom vezom.

Na kraju bloka se, kao i kod IMDN-a, spajaju destilovana obeležja, a zatim se nad spojenim obeležjima primenjuje 1×1 konvolucija koja objedinjuje informacije iz tih obeležja i ponovo formira potreban broj kanala.

Nakon nje RFDB koristi modul prostorne pažnje *Enhanced Spatial Attention* (ESA) [17]. Za razliku od CCA modula u IMDN-u, koji određuje značaj pojedinačnih kanala, ESA formira prostorno zavisne težine. Njima se različitim prostornim pozicijama unutar mapa obeležja dodeljuje različit značaj pri daljoj obradi.

Kompletna RFDN mreža sastoji se od početne konvolucije, četiri RFDB bloka i dela za objedinjavanje njihovih izlaza. Standardna konfiguracija koristi 50 početnih kanala. Nakon globalnog objedinjavanja obeležja, završni deo mreže sub-pikselnom konvolucijom povećava prostornu rezoluciju slike.



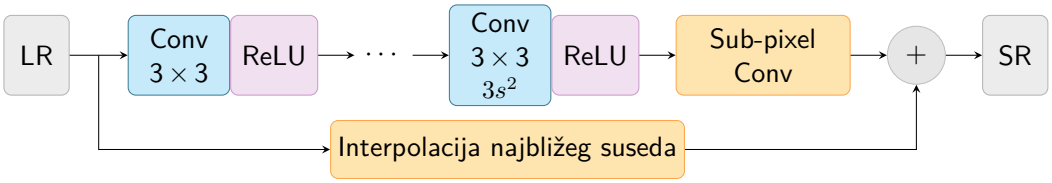
Slika 6: Arhitektura RFDN mreže.

3.6 ABPN - Anchor-Based Plain Net

ABPN [18] je nastao kao mreža za superrezoluciju namenjena mobilnim uređajima. Cilj autora bio je arhitektura koja se brzo izvršava na mobilnim akceleratorima. Idejno,

ABPN predstavlja modifikaciju VDSR-a. Kao i kod VDSR, mrežu čini niz 3×3 konvolucija, svaka osim poslednje praćena ReLU aktivacijom, bez rezidualnih veza unutar mreže, destilacije obeležja i mehanizama pažnje. Takođe, kao i VDSR, mreža ne predviđa HR sliku direktno, već rezidual u odnosu na interpoliranu ulaznu sliku. Ključna razlika je u prostoru u kome se obavlja izračunavanje. VDSR na ulazu prima već uvećanu sliku, pa sve njegove konvolucije rade na ciljnoj rezoluciji. ABPN, nasuprot tome, obrađuje originalnu LR sliku, pa se cela obrada obavlja nad s^2 puta manje piksela. Tek na kraju mreže se predviđeni rezidual uvećava sub-pikselnom konvolucijom, a zatim se sabira sa slikom uvećanom interpolacijom najbližeg suseda:

$$I^{SR} = F_\theta(I^{LR}) + I^{LR}_{nn}. \tag{18}$$



Slika 7: Arhitektura ABPN mreže.

3.7 Modifikovana arhitektura: FastEDSR

FastEDSR nije standardna arhitektura iz literature, već model razvijen u okviru ovog rada, sa ciljem da se smanji trošak inferencije, a kvalitet u što većoj meri zadrži. Idejno se zasniva na istom principu kao ABPN: predviđa se rezidual slike, sve konvolucije rade u LR prostoru, na kraju mreže se nalazi sub-pikselna konvolucija, a zatim se rezidual sabira sa interpoliranom ulaznom slikom. Razlika je u arhitekturi blokova: ABPN se može posmatrati kao VDSR premešten u LR prostor, dok je FastEDSR nastao kao olakšana verzija EDSR-a premeštena u LR prostor.

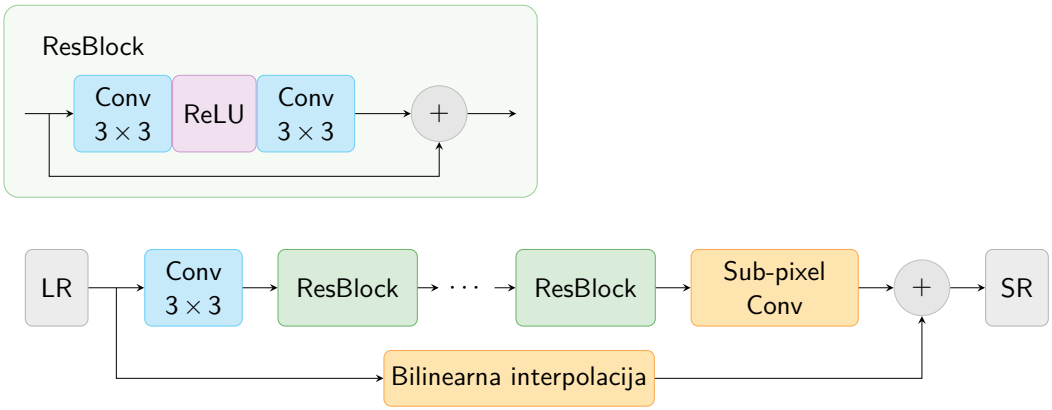
U odnosu na EDSR uklonjena su tri elementa. Najvažnija izmena je uklanjanje završne konvolucije posle sub-pikselne konvolucije. Ona radi u HR prostoru, nad s^2 puta više piksela nego ostatak mreže, pa kod malih modela čini veliki deo ukupnog izračunavanja. Bez nje konvolucija ispred operacije PixelShuffle direktno daje $3s^2$ kanala, za sve tri komponente boje, umesto $n_f s^2$ kanala obeležja, pa je i sama jeftinija. Uklonjene su i konvolucija posle tela mreže i duga rezidualna veza. Njihovu ulogu preuzima rezidual na nivou slike, jer mreža ionako predviđa samo razliku u odnosu na interpoliranu sliku.

Mreža se tako sastoji od početne konvolucije, niza rezidualnih blokova i sub-pikselne konvolucije, čime po strukturi postaje veoma slična ABPN-u. Kao i kod ABPN-a, poslednja konvolucija direktno daje rezidual, koji PixelShuffle uvećava, a sva obrada pre toga odvija se u LR prostoru. Razlikuju se samo blokovi tela mreže. ABPN koristi niz običnih konvolucija, dok FastEDSR koristi rezidualne blokove preuzete iz EDSR-a: dve 3×3 konvolucije sa ReLU aktivacijom između njih i lokalnom rezidualnom vezom, bez batch normalizacije i bez skaliranja reziduala. Lokalne rezidualne veze olakšavaju treniranje dubljih varijanti. Na ciljnoj rezoluciji obavljaju se samo interpolacija, PixelShuffle i sabiranje, pa nijedna konvolucija ne radi nad HR slikom.

Osnova na koju se dodaje rezidual dobija se bilinearnom interpolacijom, a ne interpolacijom najbližeg suseda kao kod ABPN-a:

$$I^{SR} = F_\theta(I^{LR}) + I^{LR}_{bil}. \tag{19}$$

Bilinearna osnova je glatka i nema blokovske artefakte, pa mreža treba da nauči manju korekciju prema HR slici. Izbor je motivisan i izvršavanjem modela pomoću OpenGL šejdera (poglavlje 12). Bilinearnu interpolaciju GPU obavlja hardverski pri čitanju teksture, pa se osnova u završnom šejderu dobija jednim čitanjem ulazne slike, bez dodatnog računanja (odeljak 12.3). Bilinearna interpolacija je tako najkvalitetnija osnova koja je na GPU-u dostupna bez dodatnog troška.



Slika 8: Arhitektura FastEDSR mreže.

4 Postupak treniranja modela

4.1 Skup podataka

Za treniranje modela korišćen je skup podataka DIV2K [13], koji sadrži 800 slika za treniranje i 100 slika za validaciju, rezolucije približno 2K (duža stranica slike iznosi 2040 piksela). Kvalitet modela ocenjuje se na validacionom delu skupa DIV2K.

Treniranje se sprovodi nad parovima isečaka niske (LR) i visoke (HR) rezolucije. Iz HR slike izdvaja se isečak dimenzija 96×96 piksela, a odgovarajući LR isečak dobija se bikubnim smanjivanjem, pa su LR isečci dimenzija 48×48, 32×32 i 24×24 za faktore $s = 2$, 3 i 4. Radi povećanja raznovrsnosti podataka primenjuju se nasumično isecanje, horizontalna refleksija i rotacije za 90°. Transformacije se primenjuju na HR isečak pre formiranja LR ulaza, pa su isečci u paru uvek prostorno usklađeni. Modeli obrađuju sva tri RGB kanala, a vrednosti piksela normalizuju se na opseg $[0, 1]$.

4.2 Parametri treniranja

Za treniranje se koristi ℓ_1 funkcija gubitka, odnosno srednja apsolutna greška (jednačina (17)), koja se, kao i kod EDSR-a, pokazala boljom od ℓ_2 gubitka. Optimizacija se sprovodi Adam algoritmom, sa mini-batch-om od 16 isečaka i početnom stopom učenja 10^{-4}, koja se prepolovljava nakon 300 i 600 epoha. Tokom jedne epohe iz svake trening slike izdvaja se po jedan nasumičan isečak, pa epoha odgovara 50 ažuriranja parametara, a ukupno trening traje 800 epoha, odnosno 4×10^4 ažuriranja. SRCNN, koji sporije konvergira, trenira se dvostruko duže. Model se na svakih 50 epoha ocenjuje na validacionom skupu, a kao konačan čuva se model sa najvećom vrednošću SSIM-a.

5 Metrike kvaliteta

Kvalitet rekonstruisane slike ocenjuje se metrikama koje porede generisanu SR sliku sa referentnom HR slikom. Koriste se tri metrike koje se dopunjuju: PSNR meri grešku po pikselu, SSIM sličnost lokalne strukture, a LPIPS perceptualnu sličnost.

5.1 PSNR - Peak Signal-to-Noise Ratio

PSNR je najčešće korišćena metrika u oblasti superrezolucije [19]. Definisana je na osnovu srednje kvadratne greške (MSE) između rekonstruisane slike I^{SR} i referentne slike I^{HR}:

$$\text{MSE} = \frac{1}{CHW} \sum_{c=1}^{C} \sum_{i=1}^{H} \sum_{j=1}^{W} \left(I_{cij}^{SR} - I_{cij}^{HR} \right)^2, \tag{20}$$

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right), \tag{21}$$

gde je $\text{MAX}_I = 255$ maksimalna vrednost intenziteta piksela za 8-bitne slike. Vrednosti PSNR izražene su u decibelima (dB), a više vrednosti označavaju bolji kvalitet. Mana PSNR metrike je što ne meri uvek dobro subjektivnu percepciju kvaliteta, slika može imati visok PSNR, ali izgledati zamućeno ili neprirodno.

5.2 SSIM - Structural Similarity Index

SSIM [20] je metrika dizajnirana da bolje modeluje ljudsku percepciju kvaliteta slike. Zasniva se na poređenju tri komponente između prozora $\mathbf{x}$ rekonstruisane slike I^{SR} i odgovarajućeg prozora $\mathbf{y}$ referentne slike I^{HR}: luminansa, kontrast i struktura. Prozor je mali kvadratni isečak slike (npr. 11×11 piksela) koji se pomera piksel po piksel preko cele slike, tako da se poređenje vrši lokalno, nad svakim delom slike posebno:

$$\text{SSIM}(\mathbf{x}, \mathbf{y}) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}, \tag{22}$$

gde su μ_x, μ_y lokalni srednji intenziteti, σ_x^2, σ_y^2 lokalne varijanse, σ_{xy} lokalna kovarijansa, a c_1 i c_2 male konstante za stabilizaciju. U praksi se ove statistike računaju konvolucijom sa Gausovim jezgrom, a vrednost za celu sliku dobija se usrednjavanjem po svim M prozorima:

$$\text{SSIM}(I^{SR}, I^{HR}) = \frac{1}{M} \sum_{k=1}^{M} \text{SSIM}(\mathbf{x}_k, \mathbf{y}_k). \tag{23}$$

SSIM se tako može posmatrati kao konvolucioni sloj sa fiksnim Gausovim jezgrom, praćen nelinearnom operacijom po pikselu i usrednjavanjem po celoj mapi. SSIM uzima vrednosti u intervalu $[-1, 1]$, pri čemu vrednost 1 označava savršenu sličnost, a negativne vrednosti se javljaju samo kod negativne korelacije između slika. Kod rekonstruisanih slika vrednosti su u praksi pozitivne i obično bliske jedinici.

5.3 LPIPS - Learned Perceptual Image Patch Similarity

LPIPS [21] je metrika koja meri perceptualnu sličnost u prostoru obeležja duboke konvolucione mreže pretrenirane za klasifikaciju slika (npr. AlexNet [22] ili VGG [7]). Obe slike se propuštaju kroz mrežu, aktivacije se po kanalima normalizuju, a rastojanje

se računa kao težinska ℓ_2 razlika aktivacija, usrednjena po prostornim pozicijama i sabrana po slojevima:

$$\mathrm{LPIPS}(I^{SR}, I^{HR}) = \sum_{l} \frac{1}{H_l W_l} \sum_{h,w} \left\| w_l \odot \left(\hat{\phi}^l_{hw}(I^{SR}) - \hat{\phi}^l_{hw}(I^{HR}) \right) \right\|_2^2, \tag{24}$$

gde $\hat{\phi}^l(\cdot)$ označava (po kanalima normalizovane) mape obeležja l-tog sloja, w_l naučene težine po kanalu, a $\odot$ množenje po elementima. Niže vrednosti LPIPS označavaju veću perceptualnu sličnost. Empirijski, LPIPS znatno bolje korelira sa ljudskom ocenom kvaliteta od PSNR i SSIM, pa se koristi kao dopuna pikselskim metrikama.

5.4 Način računanja metrika

Izlaz modela se pre ocene odseca na opseg $[0, 1]$ i kvantizuje na 8 bita, kao što bi bio i pri prikazu. PSNR i SSIM računaju se nad sva tri RGB kanala, bez odsecanja ivica slike, a SSIM koristi Gausov prozor veličine 11×11 sa $\sigma = 1{,}5$, pa se lokalne statistike μ, σ^2 i σ_{xy} računaju konvolucijom slike sa Gausovim jezgrom. Za LPIPS se koristi varijanta zasnovana na mreži AlexNet. Sve metrike računaju se za svaku sliku posebno, a zatim se usrednjavaju po skupu.

U literaturi o superrezoluciji PSNR i SSIM se najčešće računaju samo nad Y kanalom prostora boja YCbCr, uz odsecanje s piksela sa svake ivice slike [19]. Pošto modeli u ovom radu obrađuju i prikazuju sve kanale, ocena se vrši u RGB prostoru. Zbog toga apsolutne vrednosti nisu direktno uporedive sa vrednostima objavljenim u izvornim radovima, ali su međusobno uporedive za sve modele u ovom radu.

6 Merenje brzine i uslovi za rad u realnom vremenu

Pored kvaliteta rekonstrukcije, za primenu superrezolucije nad video-zapisima važna je i brzina obrade. Ona se može izraziti vremenom potrebnim za obradu jednog frejma, u milisekundama, ili propusnošću, odnosno brojem obrađenih frejmova u sekundi (FPS, engl. *Frames Per Second*). Ako se frejmovi obrađuju jedan za drugim, a prosečno vreme obrade jednog frejma iznosi t milisekundi, odgovarajuća propusnost je

$$\mathrm{FPS} = \frac{1000}{t}. \tag{25}$$

Niže vreme obrade, odnosno viši FPS, označava veću brzinu za zadate uslove izvršavanja. Pri sekvencijalnoj obradi, rad u realnom vremenu zahteva da se obrada svakog frejma završi pre nego što treba prikazati sledeći, odnosno da FPS obrade bude najmanje jednak broju frejmova u sekundi video-zapisa. FPS je zato praktična mera za procenu da li je model dovoljno brz za određeni video-sadržaj. Među uobičajenim vrednostima su 24 FPS, naročito kod filmova, i 30 FPS kod različitih vrsta video-zapisa, dok se za sportske prenose i sadržaje sa brzim pokretima koriste i 50 ili 60 FPS radi fluidnijeg prikaza. Za 24 FPS raspoloživo je približno 41,7 ms po frejmu, za 30 FPS oko 33,3 ms, za 50 FPS 20 ms, a za 60 FPS oko 16,7 ms.

Više vrednosti FPS-a značajne su kod video-igara, gde se frejmovi generišu u realnom vremenu. Monitori sa frekvencijom osvežavanja od 60 Hz omogućavaju prikaz do 60 različitih frejmova u sekundi, dok monitori sa višim frekvencijama, poput 120, 180 ili 200 Hz, omogućavaju fluidniji prikaz pokreta kada sistem generiše odgovarajući broj frejmova. Ako se superrezolucija primenjuje pri takvom prikazu, raspoloživo vreme za obradu dodatno se skraćuje: pri 120 FPS ono iznosi približno 8,3 ms po frejmu.

6.1 Postupak merenja brzine

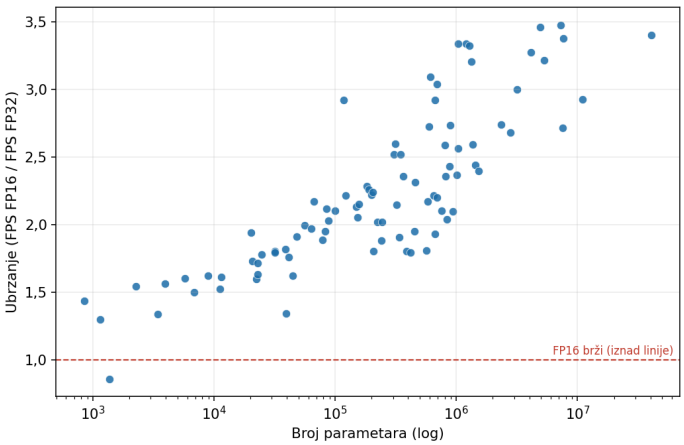
Brzina modela meri se na grafičkoj kartici NVIDIA GeForce RTX 5070. Model se pre merenja prevodi pomoću biblioteke Torch-TensorRT [23] i izvršava u FP16 preciznosti, nad ulazom veličine jednog frejma $(1 \times 3 \times H \times W)$ koji se već nalazi u memoriji grafičke kartice. Pošto vrednosti piksela ne utiču na broj operacija konvolucione mreže, kao ulaz se koristi nasumičan tenzor. Najpre se izvršava 10 zagrevajućih prolaza, a zatim se meri ukupno vreme 100 uzastopnih prolaza. Vreme t je srednje vreme jednog prolaza, a FPS se računa prema prethodnoj jednačini.

Ovako izmeren FPS obuhvata samo inferenciju modela. Dekodiranje video-zapisa, prenos podataka između centralnog procesora i grafičke kartice i priprema frejma za prikaz nisu uključeni, pa on predstavlja gornju granicu brzine koju model može postići u stvarnom plejeru. Uticaj ovih koraka na ukupno vreme obrade razmatra se u odeljku 11.4.4.

6.2 Izbor preciznosti

Težine i aktivacije mreže predstavljaju se brojevima u pokretnom zarezu (engl. *floating point*). U punoj preciznosti (FP32) svaka vrednost zauzima 32 bita, a u polupreciznosti (FP16) 16 bita. Prelazak na FP16 smanjuje tačnost zapisa, pa se može očekivati i blago pogoršanje izlaza modela, ali FP16 zapis zauzima upola manje memorije i omogućava brže izvršavanje na grafičkoj kartici. Zato je ispitano koliko se kvalitet i brzina menjaju pri prelasku sa FP32 na FP16.

Merenje je sprovedeno nad 91 konfiguracijom modela za uvećanje $\times 2$. Najveće apsolutno odstupanje između FP32 i FP16 iznosi 0,002 dB za PSNR, 0,0001 za SSIM i 0,0001 za LPIPS, što je zanemarljivo. Brzina se, s druge strane, znatno povećava: FP16 je u proseku 2,13 puta brži od FP32. Ubrzanje raste sa brojem parametara (slika 9) i za puni EDSR iznosi 3,4 puta. Od 91 testirane konfiguracije modela, samo je jedna bila sporija sa FP16. To je jedna od najmanjih konfiguracija, pa je razlika verovatno posledica šuma pri merenju.



Slika 9: Ubrzanje FP16 u odnosu na FP32 (odnos broja frejmova u sekundi) u zavisnosti od broja parametara modela, za video rezolucije 480p i uvećanje $\times 2$. Tačke iznad isprekidane linije označavaju modele kod kojih je FP16 brži.

Pošto FP16 praktično ne menja kvalitet rekonstrukcije, a znatno ubrzava inferenciju, izabrana je kao podrazumevana preciznost u svim daljim merenjima.

7 Poređenje arhitektura

Izvorni radovi uglavnom prikazuju jednu ili dve konfiguracije svake arhitekture, a efikasnost ocenjuju brojem parametara ili brojem operacija. Međutim, za primenu u realnom vremenu presudno je vreme izvršavanja, a model se ne bira prema najboljem kvalitetu koji arhitektura može da postigne, već prema kvalitetu koji postiže unutar raspoloživog vremenskog budžeta. Zato je svaka arhitektura trenirana u većem broju konfiguracija različite dubine i širine, označenih sa B/n_f, gde je B broj blokova, a n_f broj kanala konvolucionih slojeva. Cilj poređenja je da se utvrdi kako se kvalitet i brzina arhitektura menjaju sa njihovom veličinom, koje osobine arhitekture određuju brzinu i koliki se kvalitet može postići uz zadatu minimalnu brzinu.

Sva poređenja u ovom poglavlju odnose se na modele trenirane za uvećanje $\times 2$. Kvalitet je ocenjen na validacionom delu skupa DIV2K, a brzina je izmerena postupkom opisanim u poglavlju 6, za LR ulaz rezolucije 854×480 piksela (u daljem tekstu FPS 480p). Poređenje obuhvata arhitekture opisane u poglavlju 3.

7.1 Kvalitet u odnosu na interpolaciju

Pre međusobnog poređenja arhitektura potrebno je utvrditi koliko neuronski modeli poboljšavaju rezultat u odnosu na klasične interpolacijske metode, koje plejeri uobičajeno koriste za uvećanje slike. Tabela 1 prikazuje interpolacijske metode i, za svaku arhitekturu, najveću treniranu konfiguraciju. Te vrednosti ne predstavljaju najbolji kvalitet koji arhitektura može da postigne, već gornju granicu u ispitanih konfiguracija. Zato služe za procenu raspona poboljšanja u odnosu na interpolaciju, a ne za rangiranje arhitektura, koje se međusobno porede u narednim odeljcima, u zavisnosti od veličine i brzine.

Model	PSNR ↑	ΔPSNR	SSIM ↑	ΔSSIM	LPIPS ↓	ΔLPIPS
Nearest	29,25	−1,79	0,8727	−0,0204	0,1558	−0,0305
Bilinear	29,94	−1,10	0,8687	−0,0244	0,2290	+0,0427
Bicubic	31,04	0,00	0,8931	0,0000	0,1863	0,0000
Lanczos	31,51	+0,46	0,9009	+0,0078	0,2053	+0,0190
SRCNN	32,40	+1,35	0,9159	+0,0228	0,1360	−0,0503
VDSR	34,04	+3,00	0,9329	+0,0398	0,0930	−0,0933
EDSR	34,40	+3,35	0,9357	+0,0426	0,0884	−0,0979
IMDN	34,27	+3,22	0,9345	+0,0414	0,0916	−0,0947
RFDN	34,48	+3,44	0,9362	+0,0431	0,0907	−0,0956
ABPN	34,03	+2,98	0,9323	+0,0392	0,0930	−0,0933
FastEDSR	34,19	+3,15	0,9339	+0,0408	0,0916	−0,0947

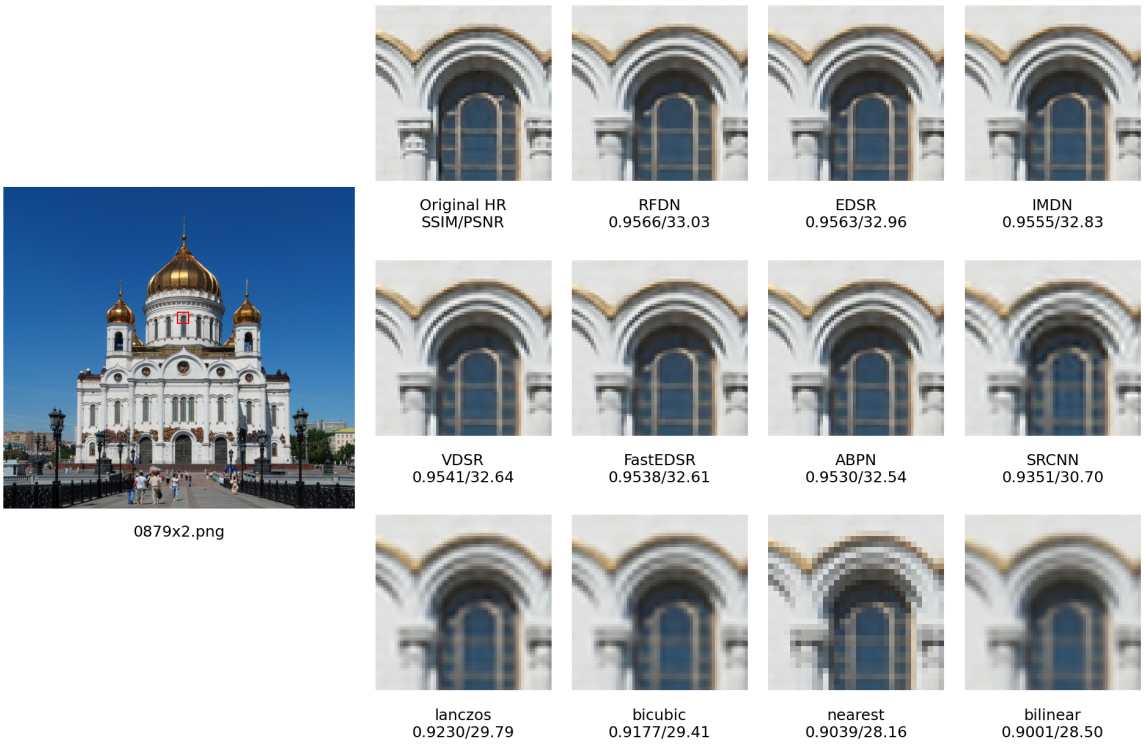
Tabela 1: Kvalitet interpolacijskih metoda i najvećih treniranih konfiguracija arhitektura. Kolone Δ prikazuju razliku u odnosu na bikubnu interpolaciju.

Svi neuronski modeli nadmašuju interpolacijske metode prema sve tri metrike. Već SRCNN, sa približno 20 hiljada parametara, povećava PSNR za 0,89 dB u odnosu na Lanczos, interpolacijsku metodu sa najvišim PSNR-om, a LPIPS smanjuje za 0,05 u odnosu na bikubnu interpolaciju. Najveće konfiguracije ostalih arhitektura povećavaju

PSNR za 2,9 do 3,4 dB u odnosu na bikubnu interpolaciju, a LPIPS smanjuju približno dvostruko, na 0,088 do 0,093. Međusobne razlike tih konfiguracija, najviše 0,46 dB, znatno su manje od njihove prednosti nad interpolacijom.

Poredak metoda zavisi i od izabrane metrike. Lanczos postiže viši PSNR i SSIM od bikubne interpolacije, ali ima lošiji LPIPS, a interpolacija najbližim susedom ima čak niži LPIPS od bikubne i bilinearne interpolacije, iako je vizuelno najgrublja. LPIPS, dakle, strože kažnjava zamućenje nego blokovske artefakte. Nijedna metrika zato sama ne opisuje kvalitet rekonstrukcije, pa se ocena zasniva na zajedničkom razmatranju metrika i vizuelnih primera.

Slika 10 prikazuje rekonstrukciju detalja sa slike 0879 iz validacionog dela skupa DIV2K. Bilinearna, bikubna i Lanczos interpolacija zamućuju konture lukova i okvira prozora, a interpolacija najbližim susedom daje stepenaste ivice. Neuronski modeli rekonstruišu oštrije i kontinualnije ivice. Izuzetak je delimično SRCNN, kod koga su konture oštrije nego kod interpolacije, ali sa vidljivim artefaktima. Razlike između ostalih neuronskih modela na ovom primeru su teško uočljive.



Slika 10: Vizuelno poređenje interpolacijskih metoda i najvećih treniranih konfiguracija arhitektura pri uvećanju $\times 2$ na slici 0879 iz skupa DIV2K.

7.2 Odnos kvaliteta i veličine modela

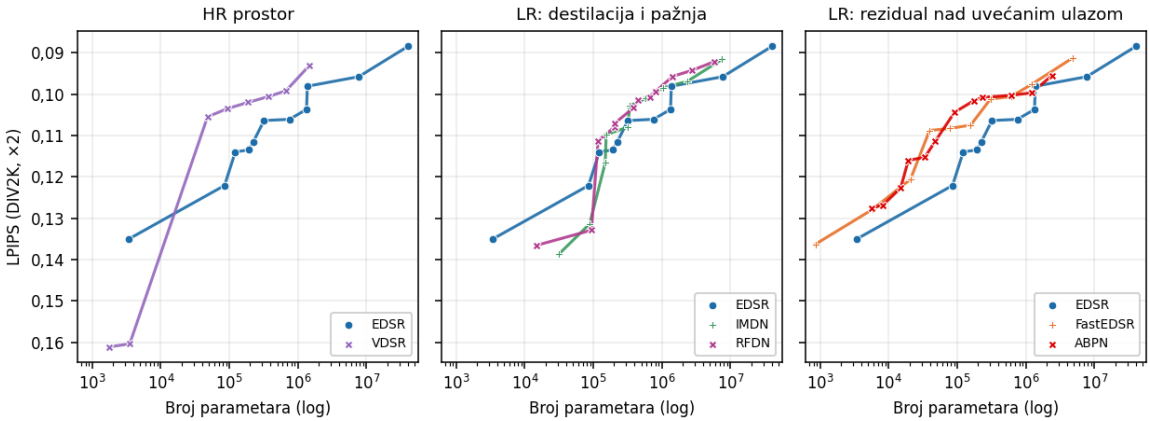
Tabela 2 izdvaja najbolje konfiguracije arhitektura po metrici, unutar svakog opsega broja parametara. Oznake K i M predstavljaju hiljade i milione parametara.

Opseg parametara	Najbolji PSNR ↑	Najbolji SSIM ↑	Najbolji LPIPS ↓
<10 K	$\text{FastEDSR}_{4/8}$ (32,74)	$\text{FastEDSR}_{4/8}$ (0,920)	$\text{ABPN}_{2/16}$ (0,127)
10 K-50 K	$\text{VDSR}_{5/32}$ (33,35)	$\text{VDSR}_{5/32}$ (0,926)	$\text{VDSR}_{5/32}$ (0,105)
50 K-200 K	$\text{VDSR}_{5/64}$ (33,64)	$\text{VDSR}_{5/64}$ (0,929)	$\text{ABPN}_{8/48}$ (0,102)
200 K-500 K	$\text{VDSR}_{10/64}$ (33,84)	$\text{VDSR}_{10/64}$ (0,931)	$\text{FastEDSR}_{4/64}$ (0,096)
500 K-1 M	$\text{RFDN}_{2/96}$ (33,90)	$\text{RFDN}_{2/96}$ (0,931)	$\text{ABPN}_{4/128}$ (0,095)
1 M-3 M	$\text{RFDN}_{4/128}$ (34,25)	$\text{RFDN}_{4/128}$ (0,934)	$\text{ABPN}_{4/256}$ (0,093)
>3 M	$\text{RFDN}_{4/256}$ (34,48)	$\text{RFDN}_{4/256}$ (0,936)	$\text{EDSR}_{32/256}$ (0,088)

Tabela 2: Najbolje konfiguracije arhitektura po metrici i opsegu broja parametara

U opsezima od 10 do 500 hiljada parametara VDSR daje najbolji PSNR i SSIM. Iznad tog opsega za PSNR i SSIM prednjači RFDN, dok za LPIPS uglavnom najbolji odnos ima ABPN.

Slika 11 grafički prikazuje odnos LPIPS metrike i broja parametara. Za svaku arhitekturu prikazan je samo njen Pareto front, odnosno konfiguracije za koje ne postoji manja konfiguracija iste arhitekture sa barem jednako dobrim kvalitetom. Da bi grafici bili pregledniji, arhitekture su raspoređene u tri panela, prema tome u kom prostoru rade konvolucije i šta mreža koristi pored njih. VDSR radi nad bikubno uvećanom slikom. IMDN i RFDN rade u LR prostoru, a njihovi blokovi pored konvolucija razdvajaju i spajaju kanale i koriste mehanizme pažnje. ABPN i FastEDSR takođe rade u LR prostoru, ali ih čine samo konvolucije, aktivacije i sabiranja, a izlaz je rezidual koji se dodaje na ulaz uvećan jednostavnom interpolacijom. EDSR je prikazan u svakom panelu kao zajednička referenca. On radi u LR prostoru sa istim jednostavnim operacijama kao ABPN i FastEDSR, ali ne predviđa rezidual nad uvećanim ulazom, pa se uz VDSR vidi šta donosi rad u LR prostoru, uz IMDN i RFDN koliko koštaju destilacija i pažnja, a uz ABPN i FastEDSR šta donosi rezidual nad uvećanim ulazom. Ista podela koristi se i na graficima u narednim odeljcima.



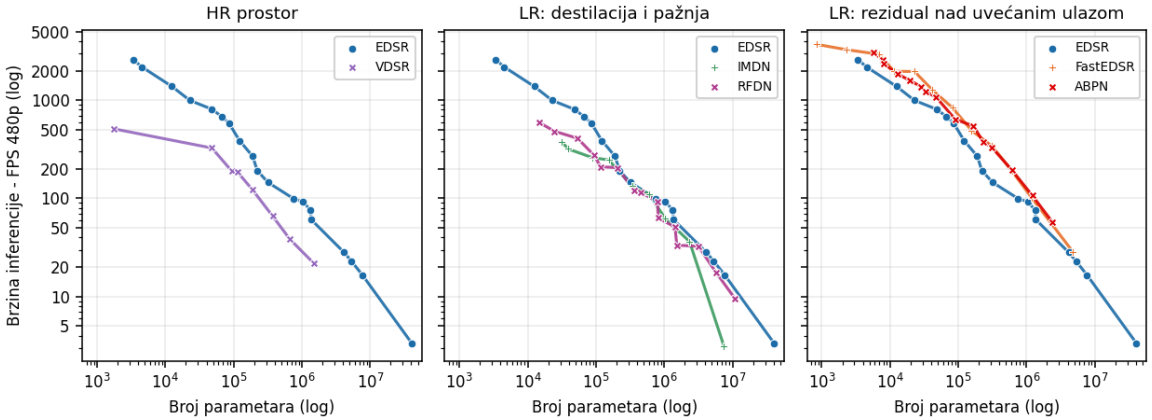
Slika 11: LPIPS kvalitet rekonstrukcije u funkciji broja parametara. LPIPS vertikalna osa je invertovana, tako da bolji kvalitet odgovara višem položaju. Horizontalna osa je logaritamska.

VDSR ima dosta dobar odnos kvaliteta i broja parametara, ali tek nakon približno 50 hiljada parametara. Rad nad uvećanom slikom ne povećava broj parametara, već samo broj izračunavanja, pa je VDSR efikasan po parametrima. U grupi sa destilacijom

i pažnjom IMDN i RFDN su međusobno približni, dok su tek od 100 hiljada parametara uporedivi sa EDSR-om. U grupi sa rezidualom nad uvećanim ulazom FastEDSR i ABPN su bolji od EDSR-a.

7.3 Odnos veličine i brzine modela

Broj parametara, međutim, ne određuje brzinu inferencije. Slika 12 prikazuje odnos brzine i broja parametara, uz Pareto front po arhitekturi, odnosno najbrže konfiguracije za datu veličinu: za konfiguraciju na frontu ne postoji konfiguracija iste arhitekture sa barem jednako mnogo parametara koja je barem jednako brza.

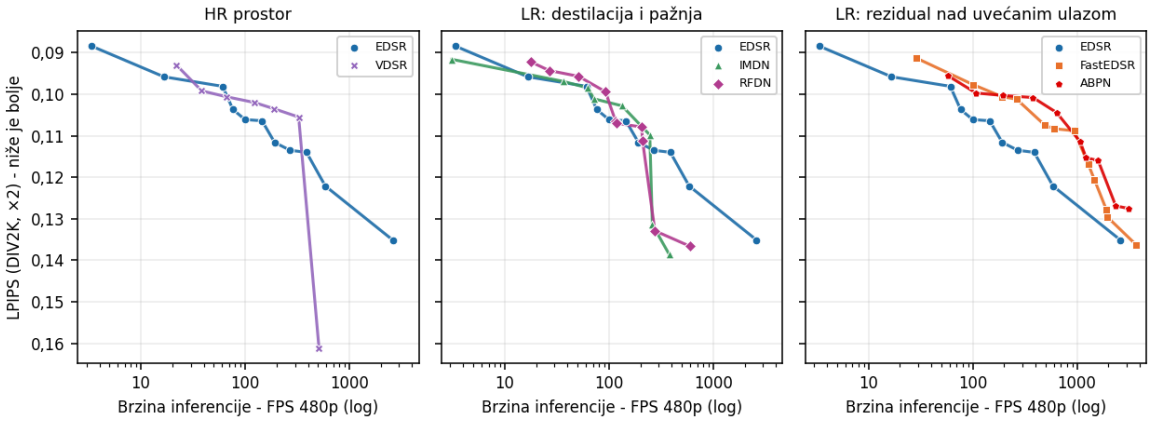


Slika 12: Brzina inferencije u odnosu na broj parametara. Obe ose su logaritamske

VDSR, koji ima najbolji odnos kvaliteta i broja parametara, ovde je najsporiji, jer sve konfiguracije izvršava nad bikubno uvećanom slikom, odnosno nad $s^2 = 4$ puta više piksela od arhitektura koje rade u LR prostoru. VDSR, IMDN i RFDN pokazuju pad brzine po parametrima nad konfiguracijama ispod 100 hiljada parametara. U grupi sa rezidualom nad uvećanim ulazom linije FastEDSR-a i ABPN-a nalaze se iznad linije EDSR-a u celom opsegu i međusobno se gotovo poklapaju. Iznad približno milion parametara razlike između arhitektura se smanjuju, jer vreme izvršavanja tada sve više određuju same konvolucije. Dobar odnos kvaliteta i broja parametara zato ne mora značiti i veliku brzinu izvršavanja, što je za rad u realnom vremenu presudno.

7.4 Odnos kvaliteta i brzine modela

Za izbor modela za video-plejer najkorisnije je pitanje koliki se kvalitet može postići uz zadatu minimalnu brzinu. Za svaki prag FPS-a zato se posmatraju svi modeli koji ga dostižu i među njima se bira najbolji. Slika 13 prikazuje odnos kvaliteta i brzine grafički. Tabele 3 i 4 ga prikazuju brojčano, kroz najbolji rezultat svake arhitekture za svaki prag.



Slika 13: Pareto front po arhitekturi: LPIPS u funkciji brzine inferencije. Vertikalna osa je invertovana, tako da bolji kvalitet odgovara višem položaju.

FPS	≥20	≥30	≥60	≥80	≥120	≥250	≥500
VDSR	0,9329	0,9307	0,9307	0,9290	0,9290	0,9265	0,9090
EDSR	0,9326	0,9326	**0,9326**	0,9278	0,9274	0,9244	0,9213
IMDN	0,9320	0,9320	0,9307	0,9293	0,9277	0,9190	—
RFDN	**0,9340**	**0,9327**	0,9311	0,9304	0,9259	0,9183	0,9114
FastEDSR	0,9333	0,9320	0,9320	**0,9320**	0,9296	**0,9296**	0,9259
ABPN	0,9319	0,9319	0,9301	0,9301	**0,9301**	0,9278	**0,9262**

Tabela 3: Najbolji SSIM ↑ koji svaka arhitektura postiže uz zadatu minimalnu brzinu. Najbolja vrednost u koloni je podebljana.

FPS	≥20	≥30	≥60	≥80	≥120	≥250	≥500
VDSR	0,093	0,099	0,101	0,102	0,102	0,105	0,161
EDSR	0,098	0,098	0,098	0,106	0,106	0,114	0,122
IMDN	0,097	0,097	0,099	0,103	0,103	0,131	—
RFDN	0,094	0,096	0,099	0,099	0,108	0,133	0,137
FastEDSR	**0,091**	0,098	**0,098**	**0,098**	0,101	0,101	0,108
ABPN	0,096	**0,096**	0,100	0,100	**0,100**	**0,101**	**0,104**

Tabela 4: Najbolji LPIPS ↓ koji svaka arhitektura postiže uz zadatu minimalnu brzinu. Najbolja vrednost u koloni je podebljana.

Iz slike i tabela izdvajaju se tri zapažanja. Prvo, kvalitet se sporo menja sa zahtevom za brzinom. Uz najmanje 30 FPS najbolji model postiže SSIM 0,9327, a uz najmanje 250 FPS SSIM 0,9296. Približno osam puta veća brzina smanjuje tako SSIM za 0,0031, što je manje od desetine poboljšanja koje modeli donose u odnosu na bikubnu interpolaciju (tabela 1). Najbolji LPIPS pri tome raste sa 0,096 na 0,101.

Drugo, uz najmanje 30 FPS SSIM najboljih konfiguracija svih arhitektura razlikuje se za najviše 0,002, ali arhitekture iz prve dve grupe ne zadržavaju kvalitet pri većim brzinama. Linije IMDN-a i RFDN-a prate EDSR do približno 200 FPS, a zatim naglo padaju, što odgovara gornjoj granici njihove brzine. Uz najmanje 250 FPS njihov

najbolji SSIM iznosi oko 0,919, naspram 0,9296 kod FastEDSR-a, a LPIPS oko 0,13, naspram 0,101, dok granicu od 500 FPS IMDN uopšte ne dostiže. VDSR je između približno 80 i 300 FPS bolji od EDSR-a, ali iznad toga naglo gubi kvalitet, jer brže konfiguracije moraju biti veoma plitke. Uz najmanje 250 FPS VDSR i EDSR zaostaju za FastEDSR-om po SSIM-u za 0,003, odnosno 0,005.

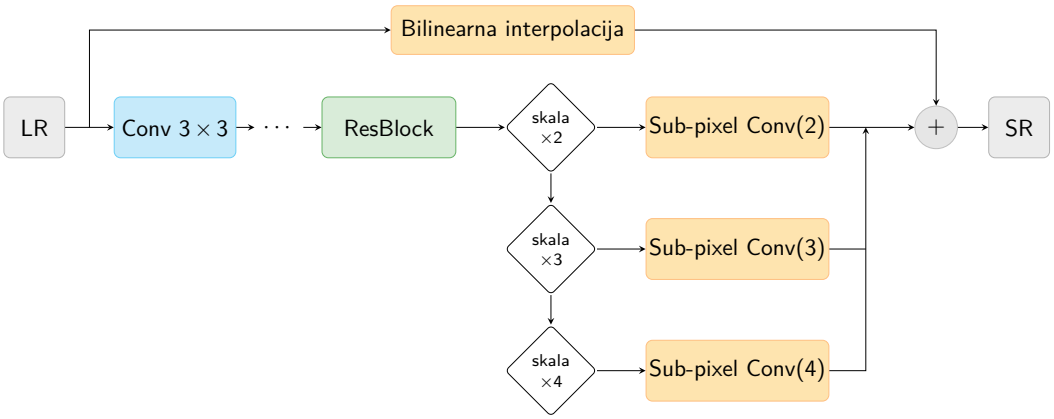
Treće, linije ABPN-a i FastEDSR-a nalaze se iznad skoro cele linije EDSR-a i međusobno su praktično izjednačene, a uz najmanje 80 FPS najbolji rezultat prema svim metrikama postižu isključivo ove dve arhitekture.

Poređenje arhitektura pokazuje da za rad u realnom vremenu najbolji odnos kvaliteta i brzine imaju jednostavne arhitekture koje rade u LR prostoru i predviđaju samo rezidual nad interpoliranim ulazom. Rad u LR prostoru smanjuje broj piksela nad kojima se izvršavaju konvolucije s^2 puta, zbog čega je VDSR, iako najefikasniji po broju parametara, najsporiji. Destilacija i pažnja kod IMDN-a i RFDN-a smanjuju broj parametara, ali dodatne operacije razdvajanja i spajanja kanala i mehanizmi pažnje produžavaju izvršavanje, pa ove arhitekture pri većim brzinama gube kvalitet. FastEDSR i ABPN objedinjuju dva svojstva: čine ih samo konvolucije, aktivacije i sabiranja, i predviđaju samo rezidual nad interpoliranim ulazom. Prvo svojstvo daje najbrže izvršavanje za dati broj parametara, a drugo omogućava da i male konfiguracije zadrže dobar kvalitet. Zato su ovakve arhitekture najpogodnije za superrezoluciju u video-plejeru.

8 Višeskalni modeli

Pored modela treniranih za pojedinačne faktore uvećanja, implementirane su i višeskalne varijante, koje u okviru jedne mreže podržavaju faktore $\times 2$, $\times 3$ i $\times 4$. Od skale zavisi samo mali deo mreže, dok je ostatak zajednički za sve faktore. Inspiracija za ovakav pristup bila je arhitektura MDSR, opisana u odeljku 3.3.2.

Kod arhitektura SRCNN i VDSR za to nije potrebna promena strukture, jer se ulazna slika unapred interpolira na ciljnu rezoluciju, pa mreža uvek radi u HR prostoru. EDSR, IMDN, RFDN, ABPN i FastEDSR, sa druge strane, uvećavaju sliku unutar same mreže sub-pikselnom konvolucijom, čiji broj izlaznih kanala i PixelShuffle zavise od faktora uvećanja. Njihove višeskalne varijante zato imaju zasebnu sub-pikselnu konvoluciju za svaki faktor, dok je ostatak mreže zajednički. Primer za višeskalni FastEDSR model dat je na slici 14.



Slika 14: Arhitektura višeskalnog FastEDSR modela.

Faktor uvećanja prosleđuje se modelu kao dodatni argument i određuje koja se sub-pikselna konvolucija izvršava, pa se pri jednoj inferenciji koristi samo grana izabrane skale. Pri treniranju se za svaki mini-batch nasumično bira jedan od tri faktora, a zatim se koriste parovi LR i HR isečaka za taj faktor. Zajednički deo mreže tako uči na primerima svih skala, dok se sub-pikselna konvolucija pojedinačne skale ažurira samo kada je ta skala izabrana.

Da bi se proverilo da li zajednički deo mreže umanjuje kvalitet, svaki višeskalni model upoređen je sa modelima iste arhitekture i konfiguracije B/n_f koji su trenirani samo za jedan faktor uvećanja, u daljem tekstu namenskim modelima. Tabela 5 prikazuje rezultate obe grupe, usrednjene po svim konfiguracijama za svaki faktor uvećanja.

Faktor	PSNR ↑		SSIM ↑		LPIPS ↓	
	namenski	multi	namenski	multi	namenski	multi
×2	33,26	**33,27**	0,9250	**0,9252**	0,1108	**0,1085**
×3	29,59	**29,67**	0,8481	**0,8497**	**0,2496**	0,2501
×4	27,73	**27,86**	0,7859	**0,7893**	0,3384	**0,3364**

Tabela 5: Prosečan kvalitet namenskih i višeskalnih modela iste konfiguracije na skupu DIV2K, po faktoru uvećanja.

Višeskalni modeli za sva tri faktora postižu u proseku približno jednak ili nešto bolji rezultat po svim metrikama. Više faktora uvećanja može se, dakle, objediniti u jednom modelu bez gubitka kvaliteta, umesto da se za svaki faktor trenira i održava zasebna mreža. To je posebno korisno u video-plejeru, gde potreban faktor zavisi od rezolucije video-zapisa i ekrana.

9 Superrezolucija JPEG-kompresovanih slika

U prethodnim eksperimentima slike niske rezolucije formirane su bikubnim smanjivanjem odgovarajućih HR slika, bez uvođenja dodatne JPEG kompresije. Referentne slike čuvane su u PNG formatu, čime se prilikom njihovog čuvanja ne uvode dodatni gubici. Međutim, u praktičnim uslovima ulazni sadržaj je često čuvan uz kompresiju sa gubicima. U tom slučaju, pored gubitka prostornih detalja usled smanjenja rezolucije, u slici su prisutni i kompresioni artefakti. Zbog toga je dodatno ispitano ponašanje modela kada su LR ulazi kompresovani JPEG algoritmom [24]. Ovakav scenario je značajan i sa stanovišta primene superrezolucije nad kompresovanim video sadržajem.

Deo modela treniran je tako što je nakon formiranja LR slike primenjena dodatna JPEG kompresija, sa uniformno nasumičnim kvalitetom u opsegu od 20 do 95, dok je odgovarajuća HR slika zadržana kao referentni izlaz. Na ovaj način model istovremeno uči povećanje prostorne rezolucije i ublažavanje artefakata nastalih kompresijom. Sličan pristup, sa JPEG kompresijom kao delom modela degradacije pri formiranju trening parova, koriste i radovi [25, 26]. Arhitektura modela pri tome nije menjana, jedina razlika u odnosu na bazne modele nalazi se u postupku pripreme podataka za treniranje.

Za evaluaciju je korišćen modifikovani test skup DIV2K, gde su LR slike dodatno JPEG-kompresovane, pri čemu je vrednost JPEG kvaliteta birana iz opsega od 20

do 100 prema ravnomernoj raspodeli. Na taj način test skup obuhvata različite stepene kompresije. Tabela 6 prikazuje rezultate baznih modela i odgovarajućih modela treniranih sa JPEG degradacijom za faktor uvećanja ×2.

Model	PSNR ↑		SSIM ↑		LPIPS ↓	
	bazni	jpeg	bazni	jpeg	bazni	jpeg
FastEDSR 4/32	28,382	**29,108**	0,804	**0,826**	**0,268**	0,280
FastEDSR 4/64	28,375	**29,332**	0,804	**0,832**	**0,262**	0,273
FastEDSR 4/128	28,388	**29,543**	0,804	**0,837**	**0,265**	0,267
FastEDSR 4/256	28,405	**29,635**	0,805	**0,839**	0,270	**0,263**
RFDN 2/128	28,366	**29,408**	0,804	**0,842**	0,265	**0,255**

Tabela 6: Poređenje JPEG modela

Rezultati pokazuju jasno poboljšanje PSNR i SSIM metrika kod modela treniranih sa JPEG degradacijom. Prosečno povećanje PSNR-a iznosi 1,022 dB, dok se SSIM u proseku povećava za oko 0,031. Ovakav rezultat pokazuje da uključivanje JPEG degradacije tokom treniranja omogućava modelu da se bolje prilagodi kompresovanim ulazima i efikasnije ublaži prisutne kompresione artefakte. Vizuelni primer se može videti na slici 15.



Slika 15: Vizuelno poređenje rekonstrukcije JPEG-kompresovane slike pri faktoru uvećanja ×2 za bazni model, model treniran sa JPEG degradacijom i bikubnu interpolaciju.

Sa druge strane, poboljšanje PSNR-a i SSIM-a nije praćeno jednakim poboljšanjem perceptualne metrike. Prosečna vrednost LPIPS-a povećava se za približno 0,0016, što predstavlja blago pogoršanje prema ovoj metrici. Vizuelnom analizom uočeno je da su rekonstruisane slike modela treniranih sa JPEG degradacijom vidljivo zamućene. Model prilikom uklanjanja kompresionih artefakata može istovremeno potisnuti deo visokofrekventnih detalja, što dovodi do blagog zamućenja i može uticati na LPIPS.

Zbog toga je ispitan i postupak u kome se tokom treniranja dodatno izoštrava referentna HR slika. LR ulaz formira se pre ovog koraka, tako da izoštravanje ne menja ulaz modela, već isključivo ciljnu sliku koju mreža treba da rekonstruiše. Na taj način model se tokom treniranja podstiče da generiše izraženije ivice i fine detalje, bez potrebe za uvođenjem zasebnog koraka izoštravanja tokom inferencije. Izoštravanje referentnih slika tokom treniranja koristi i Real-ESRGAN [26].

Pri vizuelnom poređenju izlazi ovako treniranih modela delovali su oštrije od izlaza odgovarajućih modela treniranih bez izoštravanja referentnih slika. Uticaj ovog postup-

ka dodatno je analiziran pomoću SSIM i LPIPS metrika na JPEG-kompresovanom validacionom skupu DIV2K za faktor uvećanja ×2. Rezultati su prikazani u tabeli 7, gde oznaka *jpeg* predstavlja model treniran sa JPEG degradacijom, dok *jpeg_s* označava varijantu kod koje je dodatno izoštravana referentna HR slika.

Model	SSIM ↑			LPIPS ↓		
	bazni	jpeg	jpeg_s	bazni	jpeg	jpeg_s
FastEDSR 4/32	0,804	0,826	**0,829**	**0,268**	0,280	0,270
FastEDSR 4/64	0,804	0,832	**0,835**	**0,262**	0,273	0,263
FastEDSR 4/128	0,804	0,837	**0,839**	0,265	0,267	**0,259**
RFDN 2/128	0,804	0,842	**0,842**	0,265	0,255	**0,255**

Tabela 7: Poređenje JPEG modela

U odnosu na modele trenirane samo sa JPEG degradacijom, korišćenje izoštrenih referentnih slika smanjuje LPIPS u proseku za približno 0,007, dok se SSIM dodatno povećava za približno 0,002. U poređenju sa baznim modelima, JPEG-S varijante zadržavaju znatno viši SSIM, sa prosečnim povećanjem od oko 0,032, i manjim smanjenjem LPIPS vrednosti, u proseku od 0,003, što je praktično jednako vrednosti baznih modela. Na taj način izoštravanje referentnih slika ublažava perceptualni nedostatak uočen kod modela treniranih samo sa JPEG degradacijom, bez potrebe za promenom same arhitekture mreže. Vizuelni primer na slici 16 ilustruje razliku između posmatranih pristupa.

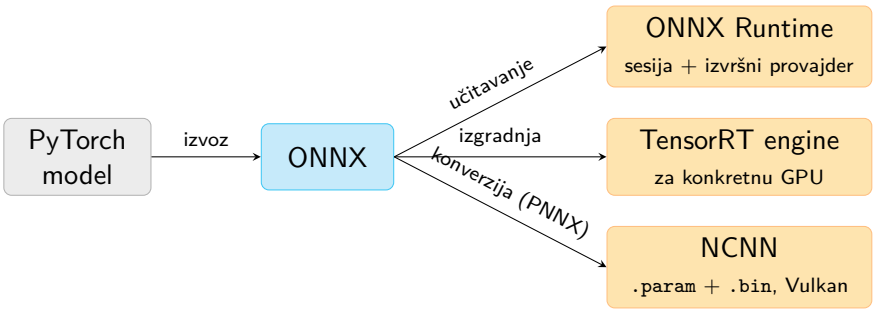


Slika 16: Vizuelno poređenje rekonstrukcije JPEG-kompresovane slike pri faktoru uvećanja ×2 za model treniran sa JPEG degradacijom i varijantu sa dodatno izoštrenom referentnom HR slikom.

10 Izvršna okruženja za inferenciju

Modeli su trenirani u okruženju PyTorch [27], ali u aplikaciji koja obrađuje videozapis model se izvršava pomoću izvršnog okruženja (engl. *runtime*). To je biblioteka koja učitava opis mreže i njene težine i izvršava operacije računskog grafa na izabranom hardveru. Izvršna okruženja razlikuju se po podržanom hardveru i operativnim sistemima, po optimizacijama koje primenjuju i po načinu pokretanja. U ovom radu korišćena su tri: ONNX Runtime, TensorRT i NCNN. Zajednička polazna tačka za sva tri je zapis modela u formatu ONNX, iz kog svako okruženje dalje priprema model na svoj način (slika 17): ONNX Runtime ga učitava neposredno, TensorRT na osnovu

njega gradi optimizovani engine za konkretnu grafičku karticu, a za NCNN se model konvertuje u sopstveni format.



Slika 17: Od treniranog modela do izvršnog okruženja.

ONNX (engl. *Open Neural Network Exchange*) [28] je otvoreni standard koji model opisuje računskim grafom, u kome čvorovi predstavljaju operacije, poput konvolucija i aktivacionih funkcija, a veze između njih tok tenzora, uz naučene težine potrebne za inferenciju. Time se treniranje odvaja od izvršavanja modela u krajnjoj aplikaciji, ONNX definiše zapis modela, a izvršava ga izvršno okruženje.

10.1 ONNX Runtime

ONNX Runtime [29] je izvršno okruženje koje učitava ONNX model, optimizuje njegov računski graf i izvršava inferenciju nad prosleđenim podacima. Prilikom inicijalizacije formira se izvršna sesija koja se koristi za inferenciju. Osim formiranja sesije, ne zahteva posebnu pripremu modela.

Podrška za različite hardverske platforme ostvaruje se preko izvršnih provajdera (engl. *Execution Providers*) [30]. Izvršni provajder je sloj unutar ONNX Runtime-a koji povezuje izvršnu sesiju sa određenim hardverskim bekendom. Pri učitavanju modela provajder određuje koje operacije ili delove računskog grafa podržava, a njihovo stvarno izvršavanje prepušta spoljnoj biblioteci, kao što su CUDA i cuDNN, TensorRT, OpenVINO ili DirectML. Te biblioteke nisu deo ONNX Runtime-a: razvijaju ih proizvođači hardvera i mogu se koristiti i samostalno, nezavisno od ONNX Runtime-a. Izuzetak je podrazumevani CPU provajder, koji sadrži sopstvene implementacije operacija. Neki od provajdera dostupnih na personalnim računarima su:

- **CPUExecutionProvider** izvršava operacije na centralnom procesoru, bez potrebe za grafičkim akceleratorom. Predstavlja osnovnu opciju za izvršavanje modela i može služiti kao rezervni provajder za operacije koje izabrani hardverski provajder ne podržava. Ovakvo izvršavanje je veoma sporo, pa se ne može koristiti za obradu video-zapisa u realnom vremenu.

- **DmlExecutionProvider** koristi Microsoft DirectML [31], biblioteku za ubrzavanje mašinskog učenja zasnovanu na DirectX 12. Omogućava izvršavanje na kompatibilnim grafičkim karticama različitih proizvođača, uključujući AMD, Intel i NVIDIA, ali samo u okviru Windows operativnog sistema.

- **TensorrtExecutionProvider** omogućava korišćenje NVIDIA TensorRT optimizacija kroz ONNX Runtime. Podržani delovi računskog grafa izdvajaju se i kompajliraju u TensorRT engine-e, dok se preostale operacije mogu izvršavati

preko drugih konfigurisanih provajdera. Time se TensorRT ubrzanje koristi uz isti interfejs izvršne sesije kao kod ostalih provajdera. TensorRT je detaljnije opisan u narednom odeljku.

- **CUDAExecutionProvider** omogućava izvršavanje na NVIDIA grafičkim karticama pomoću CUDA platforme [32] i pratećih biblioteka, među kojima je cuDNN [33] za operacije neuronskih mreža. Ovaj provajder omogućava korišćenje NVIDIA hardvera bez zasebne izgradnje TensorRT engine-a, ali zahteva kompatibilne verzije CUDA i cuDNN biblioteka.
- **OpenVINOExecutionProvider** koristi Intel OpenVINO [34] za optimizaciju i izvršavanje modela na podržanim Intel procesorima, grafičkim procesorima (integrisanim i diskretnim) i neuronskim akceleratorima (NPU).
- **MIGraphXExecutionProvider** omogućava izvršavanje na AMD grafičkim karticama korišćenjem biblioteke MIGraphX [35], koja je deo AMD ROCm platforme. Podrška je prvenstveno namenjena Linux operativnom sistemu.

10.2 TensorRT

Za izvršavanje na podržanim NVIDIA grafičkim karticama može se koristiti i TensorRT [36], alat za optimizaciju i izvršavanje neuronskih mreža sa visokim performansama. TensorRT na osnovu ONNX modela gradi optimizovani izvršni oblik mreže, *engine*, koji je prilagođen konkretnoj grafičkoj kartici na kojoj se izvršava. Glavne optimizacije koje TensorRT primenjuje su:

- Fuzija slojeva (engl. *layer fusion*) - spaja se više uzastopnih operacija (npr. konvolucija, dodavanje *bias*-a i ReLU) u jedan kernel (funkcija koja se izvršava na grafičkoj kartici), čime se izbegava upisivanje međurezultata u globalnu memoriju grafičke kartice i smanjuje broj pristupa memoriji i pokretanja kernela. Ovo se naziva *vertikalna fuzija*. Postoji i *horizontalna fuzija*, koja spaja slojeve koji primaju isti ulaz i izvršavaju istu operaciju sa sličnim parametrima (npr. više paralelnih konvolucija). Pored toga, neki slojevi mogu se u potpunosti eliminisati, na primer, kod konkatenacije prethodni slojevi mogu svoje izlaze upisati direktno na odgovarajuće pozicije u izlaznom baferu, pa zasebno kopiranje nije potrebno.
- Automatski odabir kernela - za svaki sloj postoji više mogućih implementacija (različiti algoritmi za konvoluciju, različiti CUDA kerneli). TensorRT tokom izgradnje meri dostupne implementacije na konkretnoj grafičkoj kartici i bira najbržu. Izbor zavisi i od ograničenja radne memorije i odabranog nivoa optimizacije.
- Optimizacija memorije - pošto je poznat ceo graf, TensorRT može iste memorijske bafere ponovo koristiti za tenzore čiji se periodi korišćenja ne preklapaju, čime se smanjuje ukupna potrošnja memorije.

Tokom izgradnje engine-a unapred se određuje optimizovani plan izvršavanja mreže, čime se smanjuje deo troškova upravljanja operacijama tokom inferencije. Cilj ovih optimizacija je smanjenje vremena inferencije, što je posebno značajno pri obradi videozapisa u realnom vremenu. Dobijeni engine čuva se na disku i ponovo učitava pri narednim pokretanjima. Za razliku od prenosivog ONNX zapisa, TensorRT engine vezan je za konkretno izvršno okruženje: po pravilu se može koristiti samo na grafičkoj kartici iste arhitekture i sa istom verzijom TensorRT-a sa kojom je napravljen.

TensorRT se može koristiti posredno, kroz ONNX Runtime, kao što je navedeno u prethodnom odeljku. Tada TensorRT služi kao izvršni provajder (*TensorRT Execution Provider*) koji sam gradi i kešira engine. Druga mogućnost je direktno korišćenje TensorRT-a. U ovom radu je za to napravljena posebna implementacija koja engine pokreće preko TensorRT Python API-ja, dok se PyTorch koristi za alokaciju memorije na grafičkoj kartici i upravljanje CUDA tokovima. Ovakav pristup daje više kontrole nad izvršavanjem i izbegava dodatne slojeve apstrakcije ONNX Runtime-a.

Treći način je biblioteka Torch-TensorRT [23], koja engine gradi neposredno iz PyTorch modela, bez prolaska kroz ONNX zapis. Ona je korišćena za merenje brzine modela u poglavlju 6, jer omogućava da se modeli porede bez izvoza, dok se u video-plejeru koriste ONNX modeli.

10.3 NCNN i Vulkan

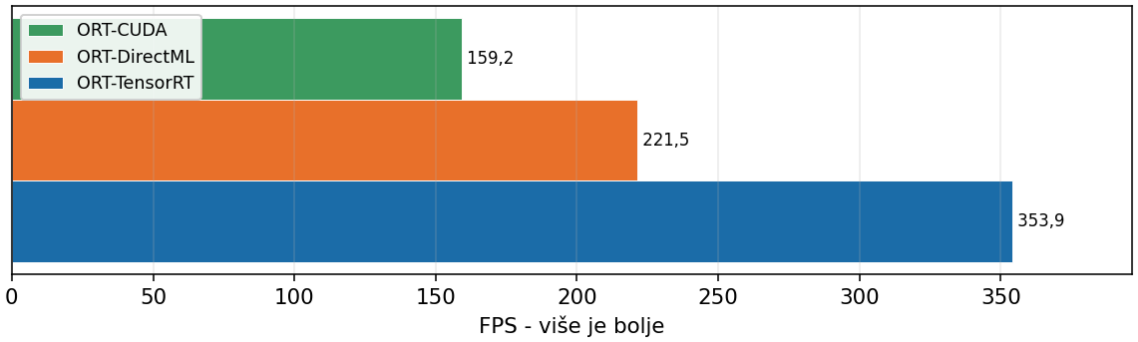
Pored prethodno opisanih izvršnih okruženja, implementirano je i izvršavanje modela pomoću biblioteke NCNN [37] uz Vulkan ubrzanje. Motivacija za ovu varijantu jeste mogućnost korišćenja grafičkih procesora različitih proizvođača na različitim operativnim sistemima. NCNN je biblioteka otvorenog koda namenjena efikasnoj inferenciji neuronskih mreža, prvobitno razvijena sa posebnim naglaskom na mobilne uređaje. Podržava izvršavanje na centralnom procesoru, kao i GPU ubrzanje preko Vulkan-a. Biblioteka obezbeđuje učitavanje modela, implementacije operacija neuronske mreže i upravljanje podacima tokom inferencije.

Vulkan [38] je programski interfejs niskog nivoa za grafiku i računanje opšte namene na GPU-u. U ovoj primeni koristi se njegova podrška za računanje, odnosno izvršavanje računskih šejdera (engl. *compute shaders*). Oni omogućavaju paralelnu obradu podataka, pa se pomoću njih mogu implementirati konvolucije, aktivacione funkcije i druge operacije neuronskih mreža. NCNN obezbeđuje odgovarajuće implementacije, tako da aplikacija ne mora ručno da piše Vulkan šejdere za svaki sloj. Vulkan je podržan na grafičkim procesorima različitih proizvođača, uključujući AMD, Intel i NVIDIA, i na više operativnih sistema.

NCNN koristi sopstveni format, pa se ONNX zapis pre inferencije konvertuje u dva fajla: `.param`, sa strukturom mreže i parametrima slojeva, i `.bin`, sa naučenim težinama. Za razliku od TensorRT engine-a, ovi fajlovi nisu vezani za konkretnu grafičku karticu, a priprema operacija za Vulkan izvršavanje obavlja se pri inicijalizaciji mreže na izabranom uređaju.

10.4 Poređenje izvršnih okruženja

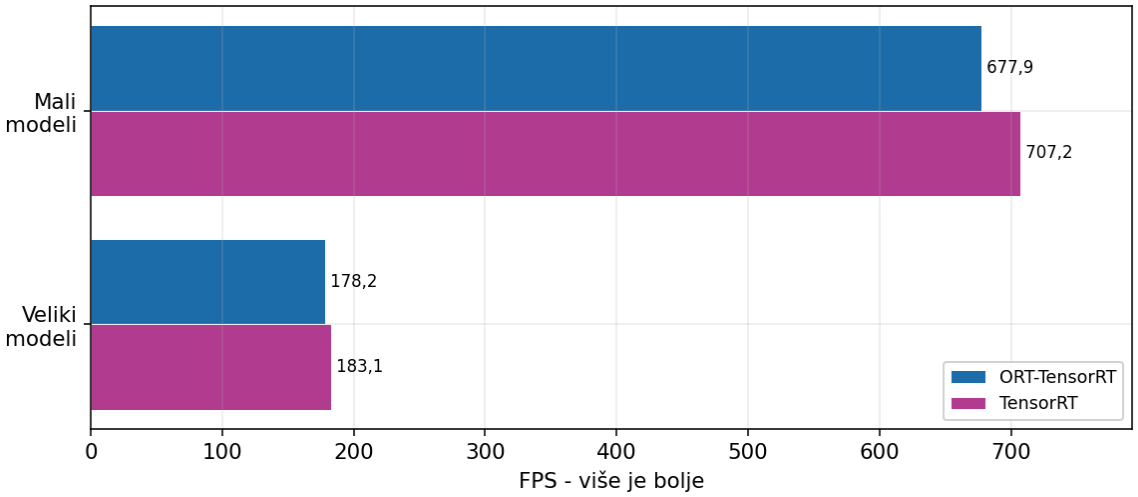
Izvršna okruženja upoređena su u demonstracionom video-plejeru (poglavlje 11), na grafičkoj kartici NVIDIA GeForce RTX 5070. Meri se trajanje koraka superrezolucije pri uvećanju $\times 2$ video-zapisa rezolucije 480p, izraženo kao broj frejmova u sekundi. Slika 18 prikazuje poređenje brzine inferencije za testiran ONNX Runtime sa provajderima CUDA, DirectML i TensorRT.



Slika 18: Brzina inferencije pri izvršavanju kroz ONNX Runtime sa različitim izvršnim provajderima (480p, ×2, prosek po modelima).

Među provajderima najbrži je TensorRT provajder, sa prosečno 353,9 FPS, što je 1,5 puta više od DirectML provajdera (221,5 FPS) i 2,2 puta više od CUDA provajdera (159,2 FPS). Optimizacije koje TensorRT primenjuje pri izgradnji engine-a donose, dakle, znatno ubrzanje i kada se TensorRT koristi kroz ONNX Runtime. DirectML je pri tome brži od CUDA provajdera.

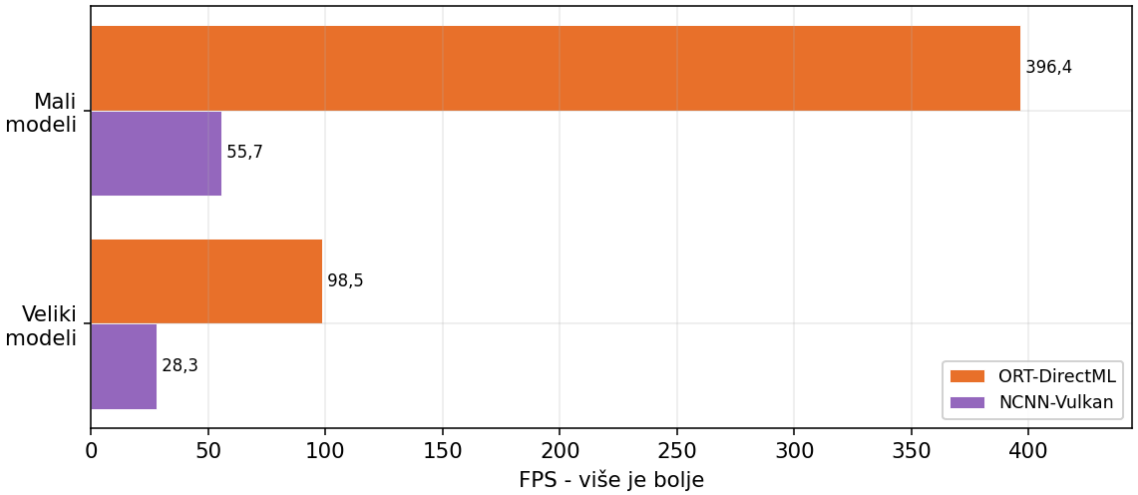
Slika 19 prikazuje poređenje brzine inferencije između korišćenja TensorRT preko ONNX izvršnog okruženja i direktnog pokretanja TensorRT engine-a.



Slika 19: Brzina inferencije pri direktnom pokretanju TensorRT engine-a i pri korišćenju TensorRT provajdera ONNX Runtime-a, za male (ispod 100 hiljada parametara) i velike modele.

Direktnim pokretanjem TensorRT engine-a postiže se veći broj frejmova u sekundi nego kroz ONNX Runtime, ali najveća razlika je kod malih modela: 707,2 naspram 677,9 FPS, odnosno oko 4,3% više. Kod velikih modela razlika je oko 2,7%. Kod malih modela samo računanje traje kratko, pa fiksni troškovi po pozivu, koje ONNX Runtime dodaje, čine veći deo ukupnog vremena.

Slika 20 prikazuje poređenje ONNX Runtime-a uz DirectML provajder sa izvršavanjem pomoću NCNN-a sa Vulkan ubrzanjem.



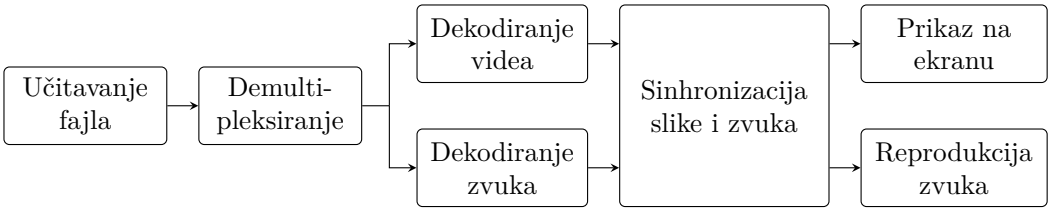
Slika 20: Brzina inferencije pri izvršavanju pomoću NCNN-a sa Vulkan ubrzanjem i ONNX Runtime-a sa DirectML provajderom, za male i velike modele.

NCNN je upoređen sa DirectML provajderom, jer su to jedina dva od izmerenih okruženja koja nisu vezana za grafičke kartice jednog proizvođača. NCNN je znatno sporiji: kod malih modela postiže 55,7 naspram 396,4 FPS, odnosno oko 7,8 puta manje, a kod velikih 28,3 naspram 98,5 FPS, oko 3,4 puta manje. Prednost NCNN-a zato nije brzina, već prenosivost, jer se, za razliku od DirectML-a, može koristiti i van Windows operativnog sistema.

Na NVIDIA grafičkim karticama najbrže je, dakle, direktno izvršavanje TensorRT engine-a, dok ONNX Runtime sa TensorRT provajderom zaostaje primetno samo kod malih modela. Za grafičke kartice drugih proizvođača pod Windows-om na raspolaganju je DirectML, koji je na korišćenoj kartici bio nekoliko puta brži od NCNN-a, a NCNN sa Vulkanom omogućava izvršavanje i na ostalim operativnim sistemima, uz manju brzinu.

11 Implementacija demonstracionog video-plejera

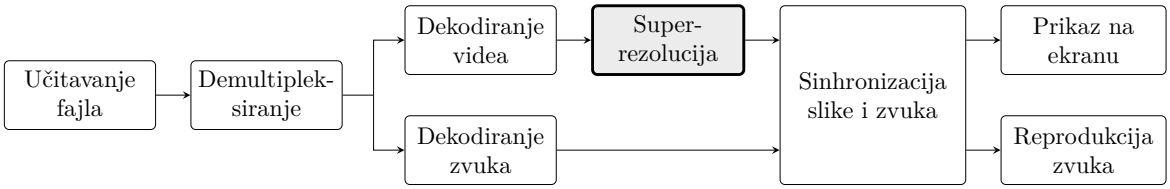
Reprodukcija video-zapisa odvija se u nekoliko koraka, prikazanih na slici 21. Plejer najpre učitava fajl, a zatim ga demultipleksira, odnosno razdvaja na video i audio tok. Svaki tok je kompresovan sopstvenim kodekom, na primer H.264 za sliku i MP3 za zvuk, pa se dekodira nezavisno od drugog. Tako se dobijaju nekompresovani frejmovi i zvučni uzorci. Na kraju plejer na osnovu vremenskih oznaka usklađuje sliku i zvuk: frejmovi se prikazuju na ekranu, a zvuk se reprodukuje preko zvučne kartice.



Slika 21: Koraci reprodukcije video-zapisa na računaru.

11.1 Mesto superrezolucije u procesu reprodukcije

Superrezolucija se u ovaj proces uključuje kao dodatni korak u video grani, između dekodiranja i prikaza. Model obrađuje vrednosti piksela, pa se ne može primeniti na kompresovani tok, već tek na dekodirani frejm. Sa druge strane, obrada mora biti završena pre prikaza, jer se na ekranu prikazuje uvećani frejm. Mesto ovog koraka u procesu reprodukcije prikazano je na slici 22.



Slika 22: Koraci reprodukcije video-zapisa sa dodatim korakom superrezolucije.

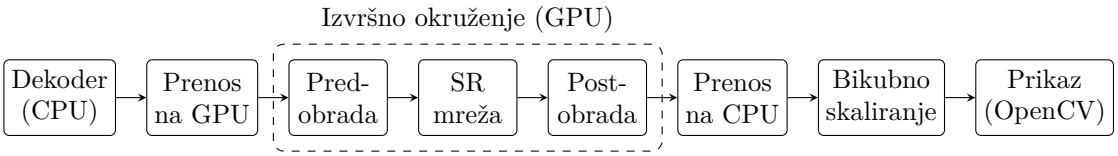
I uobičajeni plejeri uvećavaju frejm kada je rezolucija prozora ili ekrana veća od rezolucije video-zapisa. To se najčešće radi bilinearnom ili bikubnom interpolacijom na grafičkoj kartici, neposredno pre prikaza. U implementiranom plejeru uvećanje preuzima model. Pošto model uvećava celobrojnim faktorom, a odnos rezolucije ekrana i video-zapisa uglavnom nije ceo broj, plejer bira najmanji podržani faktor koji nije manji od potrebnog, a interpolacija zatim samo smanjuje izlaz modela na tačne dimenzije ekrana. Na primer, za video 854×480 na ekranu 1920×1080 potrebno je uvećanje od oko 2,25 puta, pa se bira faktor 3. Veći faktor se bira namerno, jer se pri smanjivanju rekonstruisani detalji zadržavaju, dok bi uz faktor 2 deo uvećanja ponovo obavila interpolacija. Pošto potreban faktor zavisi od rezolucije video-zapisa i ekrana, za plejer su posebno pogodni višeskalni modeli (poglavlje 8).

Uvođenje ovog koraka utiče na sinhronizaciju. Plejer prikazuje svaki frejm u trenutku određenom njegovom vremenskom oznakom, pa superrezolucija mora da se završi u okviru datog intervala T_f između dva frejma (poglavlje 6). Frejm koji nije obrađen na vreme plejer mora da preskoči ili prikaže sa zakašnjenjem, što se primećuje kao trzanje slike. Zbog toga se za plejer bira konfiguracija modela čije vreme obrade ostavlja rezervu u odnosu na T_f. Pri tome se sa intervalom T_f ne poredi samo vreme inferencije, već ukupno vreme obrade frejma. Dekodiranje, prenos podataka između procesora i grafičke kartice, završno skaliranje i prikaz takođe se odvijaju u okviru istog intervala. Kod malih modela ovi prateći koraci mogu trajati duže od same inferencije, pa se implementirane varijante plejera razlikuju po tome gde se ti koraci izvršavaju i koliko podataka pri tome prelazi između centralnog procesora i grafičke kartice. Implementirane su dve varijante: osnovna, u kojoj se dekodiranje, završno skaliranje i prikaz obavljaju na centralnom procesoru (odeljak 11.2), i zero-copy varijanta, u kojoj se svi koraci obavljaju na grafičkoj kartici, pa frejm ne napušta njenu memoriju (odeljak 11.4). Osnovna varijanta radi sa sva tri izvršna okruženja, dok zero-copy varijanta koristi TensorRT i hardverski dekoder NVDEC, pa zahteva NVIDIA grafičku karticu.

11.2 Osnovna varijanta: dekodiranje i prikaz na CPU-u

Osnovna varijanta plejera sastoji se iz četiri dela: niti za učitavanje i dekodiranje frejmova, komponente za reprodukciju zvuka, glavne petlje koja izvršava superrezoluciju i prikaz, i izvršnog okruženja za inferenciju. Za dekodiranje i prikaz koristi se

biblioteka OpenCV [39]. Ovde se koristi softversko dekodiranje frejmova, odnosno dekodiranje na centralnom procesoru (CPU). Frejm koji dekodira centralni procesor zato se mora preneti na grafičku karticu (GPU), na kojoj se izvršava superrezolucija. Uvećani frejm se zatim mora vratiti u sistemsku memoriju, kako bi se na centralnom procesoru skalirao na ciljne dimenzije i prikazao pomoću funkcije `cv2.imshow`. Tok obrade jednog frejma prikazan je na slici 23.



Slika 23: Tok obrade jednog frejma u video grani osnovne varijante plejera.

11.2.1 Učitavanje frejmova

Nit za učitavanje od dekodera traži frejmove po rednom broju, tempom koji odgovara izvornoj brzini video-zapisa. Posle svakog frejma nit na osnovu sistemskog tajmera čeka do trenutka predviđenog za sledeći. Ako dekodiranje zaostane, nit ne pokušava da nadoknadi propušteno vreme, već nastavlja od trenutnog. U osnovnoj varijanti frejmove dekodira `cv2.VideoCapture`, koji ih vraća kao NumPy niz oblika $(H, W, 3)$ tipa `uint8`, u BGR rasporedu kanala. Frejmovi se dekodiraju redom, a pri premotavanju se pozicija najpre postavlja na traženi frejm:

```
cap = cv2.VideoCapture(video_path)
fps = cap.get(cv2.CAP_PROP_FPS)

def frame(self, index):
    if index != self._next_index:
        self.cap.set(cv2.CAP_PROP_POS_FRAMES, index)
    ok, frame = self.cap.read()
    ...
```

Nit u svakom trenutku čuva samo poslednji dekodirani frejm, zajedno sa brojačem dekodiranih frejmova. Glavna petlja proverava da li se pojavio novi frejm. Ako jeste, prosleđuje ga izvršnom okruženju, a ako nije, zadržava prethodni rezultat. Ako obrada traje duže od intervala T_f, nit u međuvremenu prepisuje frejmove koji nisu stigli na obradu, pa se oni preskaču. Na taj način prikaz ne zaostaje za izvornim vremenom video-zapisa, već se smanjuje broj prikazanih frejmova u sekundi ukoliko je model spor.

11.2.2 Reprodukcija zvuka

OpenCV ne podržava rad sa zvukom, pa se audio tok pri pokretanju plejera u celosti dekodira pomoću biblioteke PyAV [40] i čuva u memoriji, a reprodukuje pomoću biblioteke `sounddevice` [41]. Tokom reprodukcije audio grana samo kopira uzorke. Biblioteka `sounddevice` poziva povratnu funkciju kad god zvučnoj kartici zatreba novi blok uzoraka:

```
def _callback(self, outdata, frames, time_info, status):
    chunk = self.data[self._pos:self._pos + frames]
    outdata[:len(chunk)] = chunk
    self._pos += frames
```

Zato audio grana ne utiče na superrezoluciju, a ista je u svim varijantama plejera. Slika i zvuk usklađuju se pri pokretanju, pauziranju i premotavanju.

11.2.3 Glavna petlja

Središnji deo petlje obrađuje samo nove frejmove i prikazuje rezultat:

```
frame, frame_counter, pos = self._reader.get_latest()

if frame_counter != last_frame_counter or last_output is None:
    output = self._produce_display(frame)
    last_output = output
    last_frame_counter = frame_counter
    needs_redraw = True

if needs_redraw:
    cv2.imshow(self.window_name, last_output)
    needs_redraw = False
key = cv2.waitKeyEx(1)
```

U osnovnoj varijanti metoda `_produce_display` poziva izvršno okruženje, a zatim rezultat bikubnom interpolacijom (`cv2.resize`) svodi na ciljne dimenzije. Dobijena slika prikazuje se funkcijom `cv2.imshow`, a sam prozor se osvežava tek u pozivu `cv2.waitKeyEx`, koji ujedno obrađuje događaje prozora i pritiske tastera. Ako se nije pojavio novi frejm niti je promenjeno stanje reprodukcije, petlja ne iscrtava sliku ponovo, već samo poziva `cv2.waitKeyEx`.

11.2.4 Izvršna okruženja osnovne varijante

Osnovna varijanta podržava sva tri izvršna okruženja opisana u poglavlju 10. Za plejer se sva tri ponašaju isto: primaju dekodirani frejm kao NumPy niz oblika $(H, W, 3)$ u BGR formatu tipa `uint8` i vraćaju uvećani frejm oblika $(sH, sW, 3)$ u istom formatu, pa je ostatak plejera nezavisan od izabranog okruženja.

ONNX Runtime prima i vraća NumPy nizove u sistemskoj memoriji, a prenose na grafičku karticu i nazad obavlja sam provajder.

```
self.session = ort.InferenceSession(str(onnx_path), providers=provider)

def __call__(self, frame):
    return self.session.run(None, {"input": frame})[0]
```

TensorRT engine radi isključivo sa adresama u memoriji grafičke kartice, pa prenose obavlja sam plejer, pomoću PyTorch tenzora. Frejm se prvo kopira u zaključanu (engl. *pinned*) sistemsku memoriju, iz koje se asinhrono prenosi na grafičku karticu, a izlaz se na isti način vraća nazad. Zaključani baferi alociraju se pri prvom frejmu i zatim ponovo koriste.

```
self.context = engine.create_execution_context()

def __call__(self, frame):
  x = self.pinned_in.copy_(torch.from_numpy(frame)).cuda(non_blocking=True)
  out = torch.empty(self.out_shape, dtype=torch.uint8, device="cuda")
  self.context.set_tensor_address("input", x.data_ptr())
  self.context.set_tensor_address("output", out.data_ptr())
  self.context.execute_async_v3(torch.cuda.current_stream().cuda_stream)
  self.pinned_out.copy_(out, non_blocking=True)
  torch.cuda.synchronize()
  return self.pinned_out.numpy()
```

NCNN, kao i ONNX Runtime, prima i vraća podatke u sistemskoj memoriji, a prenose na grafičku karticu i nazad obavlja sama biblioteka.

```
def __call__(self, frame):
    ...
    ex = self.net.create_extractor()
    ex.input("in0", mat_in)
    _, out = ex.extract("out0")
    ...
```

11.3 Predobrada i postobrada

Obrada svakog frejma obuhvata prilagođavanje ulaznih podataka formatu koji mreža očekuje, inferenciju i pripremu izlaza za prikaz. Frejmovi koje OpenCV dekoder vraća su u BGR formatu, sa celobrojnim vrednostima piksela u opsegu $[0, 255]$ i rasporedom dimenzija (H, W, C). Modeli očekuju RGB ulaz u opsegu $[0, 1]$ i rasporedu (N, C, H, W). U svrhu lake predobrade i postobrade kreirana je klasa `VideoWrapper`, koja služi kao omotač koji pre inferencije pretvara frejm u format modela, a nakon inferencije vraća rezultat u format pogodan za prikaz, tj. vrednosti se ograničavaju na $[0, 1]$, skaliraju na $[0, 255]$ i pretvaraju nazad u raspored i tip koji očekuje prikaz.

Isti omotač koristi se u svim varijantama plejera koje izvršavaju model preko TensorRT-a ili ONNX Runtime-a, a raspored ulaza i izlaza zadaje se objektom `VideoIO`. Ulaz je tipa `uint8`, u rasporedu `bgr` $(H, W, 3)$, kako ga vraća dekoder na centralnom procesoru, ili `rgb` $(3, H, W)$, kako ga vraća NVDEC (poglavlje 11.4.1). Izlaz može biti u istim rasporedima, za prikaz pomoću OpenCV-a, odnosno OpenGL-a (poglavlje 11.4.3), ili `rgb_f16`, FP16 tenzor $(1, 3, sH, sW)$ sa vrednostima u opsegu $[0, 255]$, kada izlaz modela zatim skalira grafička kartica (odeljak 11.4.3). Osnovna varijanta koristi kombinaciju `bgr`→`bgr`:

```
class VideoWrapper(torch.nn.Module):
  def __init__(self, model, io: VideoIO):
    super().__init__()
    self.model = model
    self.io = io

  def forward(self, x: torch.Tensor) -> torch.Tensor:
    # predobrada
    x = x.float() / 255.0       # uint8 [0,255] -> float [0,1]
```

```
    if self.io.input == "bgr":
      x = x[..., [2, 1, 0]].permute(2, 0, 1) # (H,W,3) BGR -> (3,H,W) RGB
    # superrezolucija
    out = self.model(x.unsqueeze(0))
    out = torch.clamp(out, 0.0, 1.0) * 255.0
    # postobrada
    if self.io.output == "rgb_f16":
      return out.to(dtype=torch.float16)     # (1,3,sH,sW) RGB
    out = out.squeeze(0)           # (3,sH,sW) RGB
    if self.io.output == "bgr":
      out = out.permute(1, 2, 0)[..., [2, 1, 0]] # (sH,sW,3) BGR
    return out.to(dtype=torch.uint8)
```

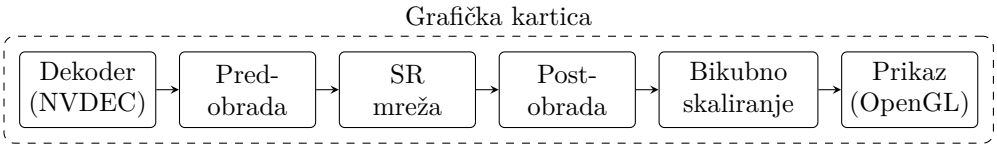
Omotač se izvozi u ONNX zajedno sa modelom, pa predobrada i postobrada postaju deo istog računskog grafa. Ovakva organizacija donosi dve prednosti. Prvo, konverzija tipa, normalizacija i permutacija kanala izvršavaju se nezavisno za svaki piksel, pa ih grafička kartica obavlja paralelno i znatno brže od centralnog procesora. Pošto su deo izvezenog grafa, TensorRT ih pri izgradnji engine-a može i spojiti sa susednim operacijama. Drugo, između procesora i grafičke kartice prenose se podaci tipa `uint8`, sa jednim bajtom po vrednosti, umesto dva bajta u FP16.

NCNN ne koristi ovaj omotač. Model se izvozi samo sa množenjem sa 255 i ograničavanjem izlaza na kraju grafa, a ostatak predobrade i postobrade obavlja se na centralnom procesoru. Funkcija `from_pixels` u jednom koraku menja raspored kanala iz BGR u RGB i raspored dimenzija iz (H, W, C) u (C, H, W), a vrednosti se zatim dele sa 255. Izlaz se pomoću OpenCV-a pretvara u `uint8` i vraća u raspored $(sH, sW, 3)$ u BGR formatu:

```
h, w = frame.shape[:2]
mat_in = ncnn.Mat.from_pixels(frame, ncnn.Mat.PixelType.PIXEL_BGR2RGB, w,
↪  h)
mat_in.substract_mean_normalize([0, 0, 0], [1 / 255.0] * 3)
...
chw = np.asarray(out)  # (3,sH,sW) RGB, [0,255]
return cv2.merge([cv2.convertScaleAbs(chw[c]) for c in (2, 1, 0)])
```

11.4 Zero-copy varijanta: dekodiranje i prikaz na GPU-u

U osnovnoj varijanti frejm po dva puta prelazi između centralnog procesora i grafičke kartice: dekodirani frejm prenosi se na grafičku karticu radi superrezolucije, a uvećani frejm vraća se u sistemsku memoriju, gde ga centralni procesor bikubno skalira i prikazuje pomoću funkcije `cv2.imshow`. Pored prenosa, centralni procesor obavlja i dekodiranje, završno skaliranje i kopiranje slike u prozor. Svi ovi troškovi mogu se izbeći ako se na grafičku karticu premeste i prvi i poslednji korak video grane. Dekodiranje tada obavlja namenski hardverski dekoder grafičke kartice, prikaz obavlja grafički API poput OpenGL-a ili Vulkana, direktno iz memorije grafičke kartice, a na grafičkoj kartici se između njih obavlja i završno bikubno skaliranje. Cela video grana ostaje na grafičkoj kartici, kako prikazuje slika 24: centralni procesor samo upravlja tokom reprodukcije i reprodukuje zvuk, a podaci o pikselima nijednom ne prolaze kroz sistemsku memoriju (engl. *zero-copy*).



Slika 24: Tok obrade frejma u zero-copy varijanti, u kojoj se dekodiranje i prikaz obavljaju na grafičkoj kartici.

U nastavku se redom opisuju hardversko dekodiranje, izvršavanje modela, završno skaliranje i prikaz pomoću OpenGL-a.

11.4.1 Hardversko dekodiranje

U osnovnoj varijanti frejm dekodira centralni procesor, nakon čega se prenosi na grafičku karticu. Međutim, većina savremenih grafičkih kartica, pored jezgara namenjenih opštim izračunavanjima, sadrži i namenske hardverske blokove za dekodiranje videa. Ovi blokovi podržavaju najrasprostranjenije video kodeke, poput H.264 i HEVC, a na novijim generacijama i AV1. Kod NVIDIA grafičkih kartica ovaj blok se naziva NVDEC [42], kod AMD kartica VCN, kod Intel kartica Quick Sync Video. Hardversko dekodiranje na grafičkoj kartici, nezavisno od proizvođača, donosi tri prednosti:

- Namenski hardverski dekoder je brži od dekodiranja na centralnom procesoru i rasterećuje ga.
- Dekoder je zaseban hardverski blok, odvojen od računskih jedinica grafičke kartice, pa se dekodiranje velikim delom odvija paralelno sa inferencijom i ne oduzima joj računske resurse.
- Dekodirani frejm ostaje u memoriji grafičke kartice, pa prenos na GPU (*Host to Device*) više nije potreban.

U nastavku se koristi NVDEC dekoder za NVIDIA grafičke kartice. Za pristup NVDEC-u korišćena je biblioteka PyNvVideoCodec [43]. Dekoder je podešen tako da dekodirane frejmove ostavlja u memoriji grafičke kartice i vraća ih u planarnom RGB formatu, kao tenzor oblika $(3, H, W)$ tipa `uint8`:

```
class NvDecoder:
  def __init__(self, video_path: str, gpu_id: int = 0):
    self.decoder = nvc.SimpleDecoder(
      str(video_path), gpu_id=gpu_id,
      use_device_memory=True,            # frames stay on the GPU
      output_color_type=nvc.OutputColorType.RGBP,  # planar RGB -> (3,H,W)
    )
    md = self.decoder.get_stream_metadata()
    self.width, self.height = int(md.width), int(md.height)
    self.fps = float(md.average_fps) or 30.0
    self.num_frames = int(md.num_frames)

  def frame(self, index: int) -> torch.Tensor:
    index = max(0, min(index, self.num_frames - 1))
    return torch.from_dlpack(self.decoder[index]).clone()
```

NVDEC sam po sebi daje frejm u formatu NV12, konverziju u RGB obavlja biblioteka, takođe na grafičkoj kartici. Dekodirani frejm predaje se PyTorch-u preko standarda DLPack [44], koji omogućava da dve biblioteke dele isti memorijski blok bez kopiranja. Međutim, plejer mora da napravi kopiju frejma (`clone`) kako dekoder ne bi prepisao taj bafer novim frejmom. Ovo kopiranje se svakako izvršava unutar memorije grafičke kartice, pa je znatno jeftinije od prenosa između procesora i grafičke kartice.

11.4.2 Izvršavanje modela

Model se izvršava preko TensorRT engine-a izgrađenog sa omotačem `VideoWrapper` (odeljak 11.3), sa ulaznim rasporedom `rgb`. Pošto dekoder već vraća RGB frejm rasporeda (C, H, W), omotač pre inferencije samo pretvara `uint8` u float, normalizuje vrednosti i dodaje dimenziju paketa. Engine se pokreće iz PyTorch-a, na zasebnom CUDA toku, a ulaz i izlaz mu se predaju kao adrese PyTorch tenzora u memoriji grafičke kartice, bez kopiranja.

```
def __call__(self, x):
  self.stream.wait_stream(torch.cuda.current_stream())
  with torch.cuda.stream(self.stream):
    x = x.to(self.in_dtype).contiguous()
    out = torch.empty(self.out_shape, dtype=self.out_dtype, device="cuda")
    self.context.set_tensor_address(self.in_name, x.data_ptr())
    self.context.set_tensor_address(self.out_name, out.data_ptr())
    self.context.execute_async_v3(self.stream.cuda_stream)
  self.stream.synchronize()
  return out
```

11.4.3 Završno skaliranje i prikaz pomoću OpenGL-a

Nad izlazom engine-a se na grafičkoj kartici obavljaju i preostali koraci video reprodukcije, završno skaliranje i prikaz. Izlaz engine-a je RGB FP16 tenzor oblika $(1, 3, sH, sW)$ sa vrednostima u opsegu $[0, 255]$, koji se bikubno smanjuje na ciljne dimenzije ekrana ukoliko je to potrebno:

```
def _downscale(self, out: torch.Tensor) -> torch.Tensor:
  if self.target_size is not None:
    out = F.interpolate(out, size=self.target_size, mode="bicubic",
    ↪  align_corners=False)
    out.clamp_(0.0, 255.0)
  return out.squeeze(0)
```

Engine vraća FP16 tenzor jer bikubna interpolacija u PyTorch-u nije podržana za celobrojne tenzore. Interpolacija nije deo engine-a jer bi se tada za svaku ciljnu veličinu morao graditi poseban engine. Nakon skaliranja vrednosti se ponovo ograničavaju na opseg $[0, 255]$, jer bikubno jezgro ima negativne delove, pa na oštrim ivicama može dati vrednosti izvan dozvoljenog opsega. U `uint8` se pretvara tek u okviru prepakovanja za teksturu, opisanog u nastavku.

U ovoj varijanti prikaz frejma na ekranu obavlja OpenGL [45]: iscrtava se pravougaonik na kome se frejm prikazuje kao tekstura. Prozor plejera otvara biblioteka GLFW [46]. CUDA i OpenGL podržavaju međusobnu razmenu resursa (engl. *CUDA–OpenGL*

interoperability). To znači da se OpenGL tekstura može registrovati kod CUDA okruženja, nakon čega CUDA u nju može direktno da upisuje. Na taj način frejm od izlaza modela do ekrana ne napušta memoriju grafičke kartice i izbegava se prenos ka procesoru i nazad.

Tekstura formata RGBA8 pravi se pri prvom frejmu, u dimenzijama tog frejma, i registruje funkcijom `cudaGraphicsGLRegisterImage`. Pri registraciji se navodi da će CUDA sadržaj teksture samo upisivati i svaki put ga ceo zameniti (`cudaGraphicsRegisterFlagsWriteDiscard`), pa drajver ne mora da čuva prethodni sadržaj.

```
tex = GL.glGenTextures(1)
GL.glBindTexture(GL.GL_TEXTURE_2D, tex)
GL.glTexImage2D(GL.GL_TEXTURE_2D, 0, GL.GL_RGBA8, w, h, 0, GL.GL_RGBA,
↪  GL.GL_UNSIGNED_BYTE, None)
GL.glBindTexture(GL.GL_TEXTURE_2D, 0)

_, resource = rt.cudaGraphicsGLRegisterImage( int(tex), GL.GL_TEXTURE_2D,
↪  rt.cudaGraphicsRegisterFlags.cudaGraphicsRegisterFlagsWriteDiscard)
```

Pozivi CUDA API-ja obavljaju se preko biblioteke `cuda-python` [47], a pozivi OpenGL-a preko biblioteke PyOpenGL [48]. Za svaki novi frejm izvršavaju se sledeći koraci:

1. Model izračuna frejm uvećane rezolucije.

   ```
   context.set_tensor_address(in_name, x.data_ptr())
   context.set_tensor_address(out_name, out.data_ptr())
   context.execute_async_v3(stream.cuda_stream)
   ```

2. Po potrebi se rezolucija smanji bikubnom interpolacijom, tako da odgovara rezoluciji ekrana.

   ```
   out = F.interpolate(out, size=self.target_size, mode="bicubic")
   out.clamp_(0.0, 255.0)
   ```

3. Planarni RGB tenzor oblika $(3, H, W)$ prepakuje se u oblik $(H, W, 4)$, u kome su komponente svakog piksela zapisane jedna do druge. Četvrta komponenta (alfa) dodaje se jer CUDA ne podržava registraciju tekstura sa tri komponente. Bafer za prepakovanje alocira se jednom i ponovo koristi.

   ```
   rgba[..., :3] = frame.permute(1, 2, 0)
   ```

4. Tekstura se mapira za CUDA pristup, čime se dobija CUDA niz (`cudaArray`) koji predstavlja njen sadržaj. Prepakovani frejm kopira se u taj niz funkcijom `cudaMemcpy2DToArray`, unutar memorije grafičke kartice. Mapiranje se zatim ukida i tekstura se vraća OpenGL-u.

   ```
   rt.cudaGraphicsMapResources(1, resource, 0)
   _, array = rt.cudaGraphicsSubResourceGetMappedArray(resource, 0, 0)
   rt.cudaMemcpy2DToArray(array, 0, 0, rgba.data_ptr(), w * 4, w * 4, h,
   ↪  rt.cudaMemcpyKind.cudaMemcpyDeviceToDevice)
   rt.cudaGraphicsUnmapResources(1, resource, 0)
   ```

5. OpenGL iscrtava pravougaonik preko oblasti iscrtavanja (`glViewport`). Šejder fragmenata za svaki piksel te oblasti uzorkuje teksturu.

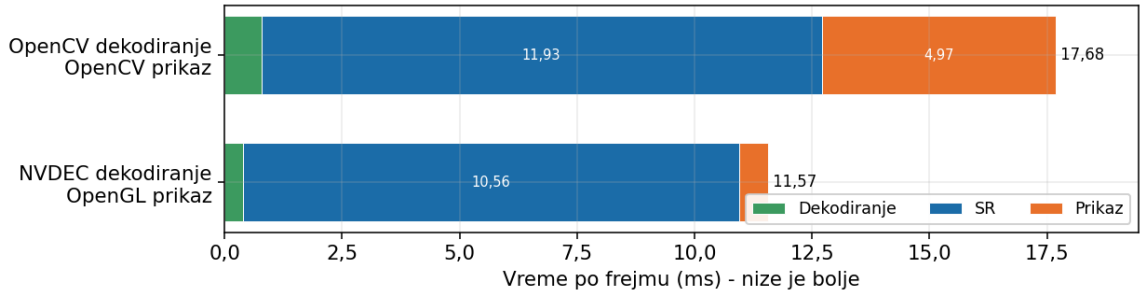
```
GL.glViewport((fb_w - vw) // 2, (fb_h - vh) // 2, vw, vh)
GL.glBindTexture(GL.GL_TEXTURE_2D, tex)
GL.glDrawArrays(GL.GL_TRIANGLE_STRIP, 0, 4)
glfw.swap_buffers(window)
```

Engine se izvršava na zasebnom CUDA toku, nakon čega procesor čeka da engine završi obradu frejma. Preostali koraci izvršavaju se redom na podrazumevanom CUDA toku, istom koji koristi PyTorch, pa se kopiranje u teksturu izvršava tek nakon skaliranja i prepakovanja, bez dodatne sinhronizacije. Pošto je bafer za prepakovanje tipa `uint8`, pri upisu skaliranog FP16 frejma u njega istovremeno se obavlja i pretvaranje u celobrojni tip, pa za to nije potrebna zasebna operacija.

11.4.4 Poređenje sa osnovnom varijantom

Da bi se utvrdilo koliko pojedini delovi obrade utiču na ukupno vreme, obrada frejma deli se na tri koraka: dekodiranje frejma, superrezoluciju (*sr*) zajedno sa prenosima podataka koje obavlja izvršno okruženje i prikaz, koji obuhvata završno skaliranje i prikaz slike. Merenje počinje posle 50 frejmova zagrevanja, a vrednosti su proseci za 500 uzastopnih frejmova. Dimenzije ciljne slike određuju se prema fiksnom referentnom ekranu rezolucije 1920×1080.

Slika 25 prikazuje trajanje pojedinih faza obrade frejma za model FastEDSR 4/128 na video-zapisu rezolucije 480p, u osnovnoj i u zero-copy varijanti.



Slika 25: Trajanje faza obrade frejma za model FastEDSR 4/128 (480p).

Hardversko dekodiranje skraćuje fazu dekodiranja sa 0,78 ms na 0,48 ms. Faza superrezolucije skraćuje se sa 11,93 ms na 10,56 ms, jer je dekodirani frejm već u memoriji grafičke kartice, pa izostaje njegov prenos, a izostaje i prenos rezultata nazad. Najveća razlika je u fazi prikaza: bikubno skaliranje i prikaz na centralnom procesoru traje 4,97 ms, a skaliranje sa kopiranjem u teksturu samo 0,67 ms. Ukupno vreme obrade frejma smanjuje se sa 17,68 ms na 11,57 ms. Ova razlika je izraženija kod manjih modela, kod kojih je vreme inferencije uporedivo sa vremenom dekodiranja i prenosa, dok kod većih modela inferencija čini najveći deo vremena obrade, pa je relativni dobitak manji.

12 Superrezolucija pomoću OpenGL šejdera

Prethodne varijante za inferenciju koriste neko izvršno okruženje za neuronske mreže i sopstveni plejer. Taj plejer je samo demonstracioni: služi za prikaz rada modela i merenje brzine, ali ne sadrži mogućnosti koje korisnici očekuju od video-plejera, kao što su podrška za titlove, plejliste ili podešavanje slike. Zato je implementiran i drugi pristup, u kome se model izvršava unutar postojećeg plejera `mpv` [49], video-plejera otvorenog koda (engl. *open source*). Plejer `mpv` omogućava da se u postupak iscrtavanja frejma ubace korisnički šejderi, pa je implementiran generator koji istreniranu mrežu prevodi u niz šejdera fragmenata (engl. *fragment shaders*), napisanih u jeziku GLSL (*OpenGL Shading Language*) [50]. Mreža se tada izvršava na grafičkoj kartici bez ikakvog izvršnog okruženja za neuronske mreže, a za dekodiranje, skaliranje i prikaz brine sam plejer.

Šejderi (od engl. *shading*, senčenje) su programi koji se izvršavaju na grafičkoj kartici. Pišu se u jeziku GLSL, a prevodi ih drajver grafičke kartice pri pokretanju programa. Šejder fragmenata računa vrednost jednog piksela izlazne teksture, a grafička kartica ga pokreće paralelno za svaki njen piksel. Pri tome dobija koordinate piksela koji računa i može da čita vrednosti sa proizvoljnih pozicija ulaznih tekstura, a kao rezultat vraća četiri vrednosti, koje odgovaraju komponentama RGBA. Šejder ne može da upisuje u druge piksele niti da čita rezultate koji se u istom izvršavanju računaju za druge piksele, pa se svi pikseli izlazne teksture računaju nezavisno jedni od drugih.

Na isti način radi i konvolucioni sloj, u kome se svaki izlazni piksel računa nezavisno od ostalih, iz okoline odgovarajućeg piksela ulazne mape obeležja. Osnovna ideja je zato da se svaki konvolucioni sloj mreže izvrši pomoću šejdera. Mape obeležja čuvaju se u teksturama. Šejder čita teksture prethodnog sloja i upisuje rezultat u novu teksturu, koju zatim čita sledeći sloj. Jedno takvo izvršavanje šejdera naziva se prolaz (engl. *render pass*).

12.1 Korisnički šejderi plejera mpv

Korisnički šejderi učitavaju se opcijom `--glsl-shaders` [51]:

```
mpv --glsl-shaders="~~/shaders/FastEDSR.glsl" video.mp4
```

Jedan fajl može da sadrži više šejdera. Svaki šejder počinje zaglavljem od direktiva oblika `//!NAZIV vrednost`, koje određuju kada se šejder izvršava, koje teksture čita i gde se upisuje rezultat. Iza zaglavlja sledi GLSL kod sa funkcijom `vec4 hook()`, čija se povratna vrednost upisuje u trenutni piksel izlazne teksture. Jednostavan šejder koji posvetljava sliku izgleda ovako:

```
//!DESC Primer: posvetljavanje slike
//!HOOK MAIN
//!BIND MAIN
//!SAVE MAIN
vec4 hook() {
    vec4 color = MAIN_texOff(vec2(0.0, 0.0));
    return vec4(clamp(color.rgb * 1.2, 0.0, 1.0), color.a);
}
```

Direktive imaju sledeću ulogu:

- `//!DESC` - opis šejdera.
- `//!HOOK MAIN` - tačka u procesu iscrtavanja na koju se šejder „kači“. Dekodirani frejm je najčešće u formatu YUV, pa ga `mpv` najpre svodi na punu rezoluciju i pretvara u RGB. Tako dobijena slika, pre skaliranja na veličinu prozora, naziva se `MAIN`, pa šejderi dobijaju upravo sliku niske rezolucije u opsegu $[0, 1]$.
- `//!BIND ime` - vezuje teksturu `ime` (slika `MAIN` ili tekstura koju je sačuvao neki raniji šejder) tako da šejder može da je čita.
- `//!SAVE ime` - rezultat šejdera se čuva u novu teksturu sa datim imenom, koju mogu da vežu kasniji šejderi. Bez ove direktive rezultat zamenjuje sliku na koju je šejder zakačen, kao u primeru iznad.
- `//!WIDTH` i `//!HEIGHT` - dimenzije izlazne teksture, zadate izrazom u postfiksnoj notaciji nad dimenzijama drugih tekstura. Bez njih izlazna tekstura ima dimenzije slike na koju je šejder zakačen.
- `//!COMPONENTS` - broj komponenti izlazne teksture, najviše četiri (RGBA).
- `//!WHEN` - uslov, takođe zadat izrazom u postfiksnoj notaciji, da li se šejder pokreće.

Za svaku teksturu `ime` koju šejder vezuje direktivom `//!BIND`, `mpv` automatski definiše pomoćne funkcije i promenljive: `ime_tex(pos)` čita teksturu na normalizovanoj koordinati `pos` $\in [0, 1]^2$, `ime_texOff(off)` čita piksel pomeren za `off` piksela u odnosu na trenutni, `ime_pos` je normalizovana koordinata trenutnog piksela u toj teksturi, a `ime_size` dimenzije teksture u pikselima. Alokaciju tekstura i okvirnih bafera, prevođenje šejdera i njihovo izvršavanje takođe obavlja `mpv`.

Implementiran je generator šejdera za arhitekturu FastEDSR, čija je struktura dovoljno jednostavna da se svaki sloj direktno preslika u šejdere. Generator učitava težine iz sačuvanog modela, prolazi redom kroz slojeve i sve šejdere upisuje u jedan fajl sa ekstenzijom `.glsl`.

12.2 Prevođenje konvolucija u šejdere

Glavna ideja je čuvati mape obeležja u teksturama. Tekstura u formatu RGBA čuva četiri vrednosti po pikselu, pa jedna tekstura može da sadrži četiri mape obeležja. Izlaz sloja sa C kanala se zato predstavlja sa $G = \lceil C/4 \rceil$ tekstura, koje ćemo nazivati grupama. Ove grupe ne treba mešati sa grupama kod grupisane konvolucije (engl. *grouped convolution*), kod koje se kanali dele u grupe koje se obrađuju međusobno nezavisno. Ovde je podela na grupe samo način smeštanja kanala u teksture, a svaki izlazni kanal i dalje zavisi od svih ulaznih kanala. Ako C nije deljivo sa 4, preostale komponente poslednje grupe popunjavaju se nulama.

Šejder fragmenata se izvršava nezavisno za svaki piksel izlazne teksture i računa vrednost samo tog piksela, pri čemu može da čita proizvoljne piksele ulaznih tekstura. Zato konvoluciju posmatramo iz ugla jednog izlaznog piksela (i, j). Za konvoluciju 3×3 potrebno je izračunati težinsku sumu njegove 3×3 okoline preko svih ulaznih kanala, pa je vrednost izlaznog piksela:

$$Y_{o,i,j} = b_o + \sum_{c=0}^{C_{\text{in}}-1} \sum_{i_k=-1}^{1} \sum_{j_k=-1}^{1} w_{o,c,i_k+1,j_k+1} \, X_{c,\, i+i_k,\, j+j_k}, \tag{26}$$

gde su

- $o \in \{0, \ldots, C_{\text{out}} - 1\}$ i $c \in \{0, \ldots, C_{\text{in}} - 1\}$ - indeksi izlaznog i ulaznog kanala
- $i_k, j_k \in \{-1, 0, 1\}$ - pomaci unutar 3×3 okoline piksela (i, j).
- $Y_{o,i,j}$ - vrednost o-te izlazne mape obeležja na poziciji (i, j)
- $X_{c,i,j}$ - vrednost c-te ulazne mape obeležja na poziciji (i, j), pri čemu je $X_{c,i,j} = 0$ za pozicije van slike (dopunjavanje nulama)
- w_{o,c,i_k+1,j_k+1} - težina jezgra koje povezuje ulazni kanal c sa izlaznim kanalom o, za pomak (i_k, j_k) u odnosu na centar jezgra
- b_o - pomeraj (engl. *bias*) izlaznog kanala o

Ako sve kanale jednog piksela posmatramo kao vektor, jednačina (26) postaje

$$\mathbf{y}_{i,j} = \mathbf{b} + \sum_{i_k=-1}^{1} \sum_{j_k=-1}^{1} \mathbf{W}_{i_k,j_k} \, \mathbf{x}_{i+i_k,\, j+j_k}, \tag{27}$$

gde su:

- $\mathbf{y}_{i,j} \in \mathbb{R}^{C_{\text{out}}}$ vektor izlaznih kanala na poziciji (i, j)
- $\mathbf{x}_{i,j} \in \mathbb{R}^{C_{\text{in}}}$ vektor ulaznih kanala na poziciji (i, j)
- $\mathbf{b} \in \mathbb{R}^{C_{\text{out}}}$ vektor pomeraja
- $\mathbf{W}_{i_k,j_k} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$ matrica težina za jednu poziciju kernela, sa elementima $(\mathbf{W}_{i_k,j_k})_{o,c} = w_{o,c,i_k+1,j_k+1}$

Konvolucija u jednom pikselu je, dakle, zbir devet proizvoda matrice i vektora.

Pošto šejder za svaki piksel upisuje samo jednu RGBA vrednost, jedan šejder računa samo jednu izlaznu grupu od 4 kanala, pa za izlaznu grupu q važi

$$\mathbf{y}^{(q)}_{i,j} = \mathbf{b}^{(q)} + \sum_{g=0}^{G_{\text{in}}-1} \sum_{i_k=-1}^{1} \sum_{j_k=-1}^{1} \mathbf{W}^{(q,g)}_{i_k,j_k} \, \mathbf{x}^{(g)}_{i+i_k,\, j+j_k}, \tag{28}$$

gde su $\mathbf{y}^{(q)}_{i,j}, \mathbf{b}^{(q)} \in \mathbb{R}^4$ izlazni kanali, odnosno pomeraji sa indeksima $4q, 4q+1, 4q+2, 4q+3$, a $\mathbf{W}^{(q,g)}_{i_k,j_k} \in \mathbb{R}^{4\times4}$ blok matrice $\mathbf{W}_{i_k,j_k}$ koji čine vrste $4q, 4q+1, 4q+2, 4q+3$ i kolone $4g, 4g+1, 4g+2, 4g+3$.

Svaki sabirak u (28) je proizvod 4×4 matrice i vektora dužine 4, što u jeziku GLSL odgovara jednoj operaciji `mat4 * vec4`. Sve tri sume u (28) imaju unapred poznat broj članova, pa se petlje razmotavaju već pri generisanju šejdera, a težine se upisuju direktno u izvorni kod kao konstante. Za jednu ulaznu grupu ($g = 0$) dobija se:

```
vec4 result = vec4(0.0);
result += mat4(0.1769, 0.1084, ...) * get_0(-1,-1);
result += mat4(...) * get_0(-1,0);
result += mat4(...) * get_0(-1,1);
result += mat4(...) * get_0(0,-1);
result += mat4(...) * get_0(0,0);
result += mat4(...) * get_0(0,1);
result += mat4(...) * get_0(1,-1);
result += mat4(...) * get_0(1,0);
result += mat4(...) * get_0(1,1);
result += vec4(-0.0151, -0.1706, -0.0361, -0.0863);
return result;
```

Makro `get_g(dx, dy)` čita piksel teksture grupe g pomeren za `dx` kolona i `dy` vrsta u odnosu na trenutni piksel, tj. vraća $\mathbf{x}^{(g)}_{i+\mathtt{dy},\, j+\mathtt{dx}}$. Definiše se preko funkcije `_texOff` koju obezbeđuje `mpv`, na primer `#define get_0(x, y) conv_net_0_p0_texOff(vec2(x, y))`. Kada sloj ima više ulaznih grupa, ovih devet redova se ponavlja za svaku od njih (`get_1`, `get_2`, ...).

Za svaku izlaznu grupu generiše se poseban šejder, pa se jedan konvolucioni sloj sa C_{in} ulaznih i C_{out} izlaznih kanala prevodi u $G_{\text{out}} = \left\lceil \frac{C_{\text{out}}}{4} \right\rceil$ šejdera.

Teksture se imenuju po sloju mreže i grupi: izlazna grupa q sloja `net.1.layers.0` čuva se u teksturi `conv_net_1_layers_0_p`q. Na primer, drugi konvolucioni sloj prvog rezidualnog bloka sa $C_{\text{in}} = C_{\text{out}} = 32$ ima $G_{\text{in}} = 8$ ulaznih grupa, pa šejder za njegovu izlaznu grupu $q = 0$ izgleda ovako:

```
//!DESC net_1_layers_2 Conv3x3 part 0 chunk 0-8
//!HOOK MAIN
//!BIND conv_net_1_layers_0_p0
...
//!BIND conv_net_1_layers_0_p7
//!BIND conv_net_0_p0
//!SAVE conv_net_1_layers_2_p0
//!WIDTH conv_net_1_layers_0_p0.w
//!HEIGHT conv_net_1_layers_0_p0.h
//!COMPONENTS 4
//!WHEN OUTPUT.w MAIN.w / 1.2 > OUTPUT.h MAIN.h / 1.2 > +
#define get_0(x_off, y_off) conv_net_1_layers_0_p0_texOff(vec2(x_off,
↪  y_off))
...
#define get_7(x_off, y_off) conv_net_1_layers_0_p7_texOff(vec2(x_off,
↪  y_off))
vec4 hook() {
  vec4 result = vec4(0.0);
  result += mat4(0.0132, -0.0473, ...) * get_0(-1,-1);
  result += mat4(...) * get_0(-1,0);
  ... // 9 * 8 = 72 ovakva reda
  result += vec4(0.0021, ...);  // bias
  result += conv_net_0_p0_texOff(vec2(0.0, 0.0));   // rezidualna veza
  return result;
}
```

Direktive `//!BIND` vezuju ulazne teksture `conv_net_1_layers_0_p0`, ..., `_p7`, po jednu za svaku ulaznu grupu $\mathbf{x}^{(g)}$, i teksturu `conv_net_0_p0` sa ulazom rezidualnog bloka (ovde izlaz početne konvolucije) za istu izlaznu grupu q, koja se na kraju dodaje rezultatu (rezidualna veza). Rezultat se direktivom `//!SAVE` čuva u teksturu `conv_net_1_layers_2_p0`, koju vezuju šejderi narednog bloka. Sve mape obeležja imaju rezoluciju ulazne slike, pa se dimenzije izlazne teksture preuzimaju od ulazne teksture, a izlazna tekstura ima četiri komponente, jer čuva jednu grupu.

U prvom konvolucionom sloju rezidualnog bloka umesto rezidualne veze na kraju stoji aktivaciona funkcija ReLU, `result = max(result, 0.0);`. Početna konvolucija umesto tekstura sa međurezultatima vezuje samo sliku `MAIN`.

Jedan šejder može istovremeno da veže ograničen broj tekstura. Kod širih slojeva ulazne grupe se zato dele u delove od najviše $k = 8$ grupa. Svaki deo

obrađuje poseban šejder, koji svoju delimičnu sumu čuva u pomoćnu teksturu (npr. `conv_net_1_layers_2_p0_c0`). Naredni deo tu teksturu vezuje, očitava je i na nju dodaje svoj doprinos.

12.3 Završni prolaz: PixelShuffle i bilinearna osnova

Završni prolaz radi u izlaznoj rezoluciji i objedinjuje operaciju PixelShuffle i sabiranje sa bilinearno uvećanim ulazom. Za piksel izlazne slike na položaju (x, y) određuje se položaj unutar bloka $s \times s$, $idx = (x \bmod s) + s\,(y \bmod s)$, i odgovarajući piksel niske rezolucije $(\lfloor \frac{x}{s} \rfloor, \lfloor \frac{y}{s} \rfloor)$. Vrednost boje c uzima se iz kanala $c\,s^2 + idx$ poslednje konvolucije, što je upravo preslikavanje koje obavlja PixelShuffle. Za uvećanje $\times 2$ poslednja konvolucija ima 12 kanala, odnosno tri teksture, pa je kanal $c\,s^2 + idx$ upravo komponenta idx teksture `pc`:

```
//!DESC PixelShuffle x2
//!HOOK MAIN
//!BIND MAIN
//!BIND conv_upscale_block_2_0_p0
//!BIND conv_upscale_block_2_0_p1
//!BIND conv_upscale_block_2_0_p2
//!SAVE MAIN
//!WIDTH conv_upscale_block_2_0_p0.w 2 *
//!HEIGHT conv_upscale_block_2_0_p0.h 2 *
//!COMPONENTS 4
//!WHEN OUTPUT.w MAIN.w / 1.2 > OUTPUT.h MAIN.h / 1.2 > + OUTPUT.w MAIN.w /
↪  2.2 < OUTPUT.h MAIN.h / 2.2 < + *
vec4 hook() {
  ivec2 pos = ivec2(gl_FragCoord.xy);
  ivec2 sub = pos & 1;                        // (x mod 2, y mod 2)
  int idx = sub.x + (sub.y << 1);             // polozaj u bloku 2x2

  vec2 source_pos = floor(pos / 2.0) + 0.5;   // centar LR piksela
  vec2 tex_coord = source_pos / conv_upscale_block_2_0_p0_size;

  vec4 p0 = conv_upscale_block_2_0_p0_tex(tex_coord);    // R
  vec4 p1 = conv_upscale_block_2_0_p1_tex(tex_coord);    // G
  vec4 p2 = conv_upscale_block_2_0_p2_tex(tex_coord);    // B

  vec3 res = vec3(p0[idx], p1[idx], p2[idx]);
  vec3 final_output = MAIN_tex(MAIN_pos).rgb + res; // bilinear + rezidual
  return vec4(clamp(final_output, 0.0, 1.0), 1.0);
}
```

Promenljiva `gl_FragCoord.xy` sadrži koordinate centra trenutnog piksela izlazne teksture. Koordinata odgovarajućeg piksela niske rezolucije deli se dimenzijama teksture (`_size`), jer funkcija `_tex` očekuje normalizovane koordinate. Slika `MAIN` je u ovom trenutku još uvek niske rezolucije, pa se čitanjem na koordinati `MAIN_pos` trenutnog izlaznog piksela dobija bilinearno uvećan ulaz, koji se sabira sa rekonstruisanim rezidualom. Rezultat se ograničava na opseg $[0, 1]$ i direktivom `//!SAVE MAIN` postaje nova slika `MAIN`, s puta veće rezolucije. Za uvećanja $\times 3$ i $\times 4$ poslednja konvolucija ima 27, odnosno 48 kanala (7, odnosno 12 tekstura), pa kanali boja više nisu poravnati sa tek-

sturama. Završni šejder tada čita sve teksture i kanal $c\,s^2 + idx$ bira kao komponentu $(c\,s^2 + idx) \bmod 4$ teksture $\lfloor (c\,s^2 + idx)/4 \rfloor$.

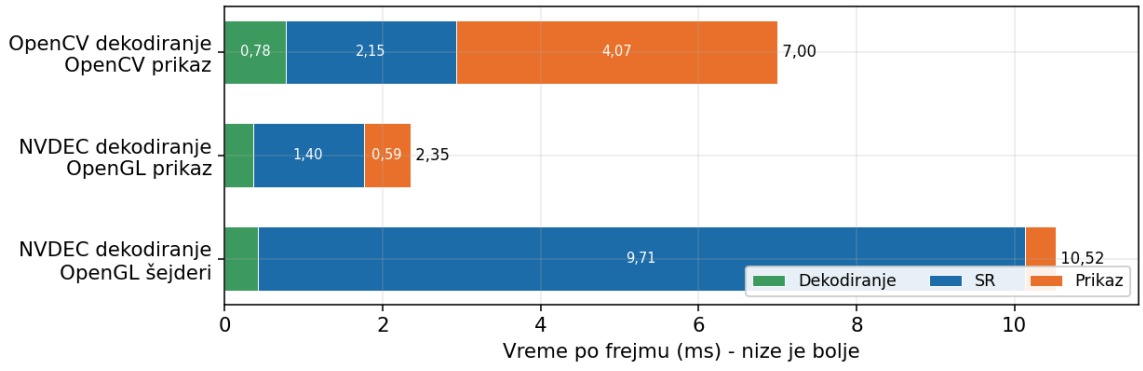
Višeskalni model ima zajedničko telo i posebnu konvoluciju za uvećanje za svaki faktor $s \in \{2, 3, 4\}$. Generator zato u isti fajl upisuje šejdere tela i sve tri grane za uvećanje, a direktiva `//!WHEN` određuje koja se grana izvršava.

Posle šejdera, slika `MAIN` ima rezoluciju s puta veću od frejma, ali ne nužno jednaku rezoluciji prozora. Preostalo prilagođavanje veličini prozora obavlja `mpv` svojim filtrom za skaliranje, pa se, kao i u demonstracionom plejeru, izlaz modela samo malo uvećava ili umanjuje do konačne veličine.

12.4 Ograničenja

Najveće ograničenje je broj prolaza, koji brzo raste sa veličinom mreže. Broj prolaza jednog sloja raste sa brojem izlaznih grupa i brojem delova na koje se dele ulazne grupe. Kada se broj kanala udvostruči, udvostručuju se obe vrednosti, pa se broj prolaza po sloju učetvorostručuje. Sloj sa 64 ulazna i izlazna kanala zahteva 32 prolaza, a sloj sa 128 kanala već 128 prolaza, pri čemu svaki novi sloj dodaje još toliko. Kod širokih i dubokih modela šejdera je zato toliko mnogo da njihovo izvršavanje u realnom vremenu praktično nije moguće.

I kod manjih modela izvršavanje pomoću šejdera je sporije nego pomoću TensorRT-a. Slika 26 prikazuje trajanje faza obrade frejma za model FastEDSR 4/32 na video-zapisu rezolucije 480p, u osnovnoj i zero-copy varijanti demonstracionog plejera i pri izvršavanju pomoću šejdera. Šejderi su za ovo merenje izvršavani u demonstracionom plejeru, uz NVDEC dekodiranje, a faza superrezolucije obuhvata i prenos ulaznog frejma u OpenGL teksturu. Superrezolucija pomoću šejdera traje 9,71 ms, oko sedam puta duže nego u zero-copy varijanti (1,40 ms). Razlog je to što konvolucije u šejderima fragmenata ne koriste tenzorska jezgra, a svaki prolaz upisuje međurezultate u zasebnu teksturu u memoriji grafičke kartice i ponovo čita sve ulazne teksture. Ukupno vreme od 10,52 ms je ipak dovoljno za reprodukciju u realnom vremenu, jer je i pri 60 frejmova u sekundi na raspolaganju 16,7 ms po frejmu.



Slika 26: Trajanje faza obrade frejma za model FastEDSR 4/32 (480p).

Prednost ovog pristupa je u tome što nije potrebno nikakvo izvršno okruženje za neuronske mreže, ni poseban plejer: dovoljan je jedan `.glsl` fajl i `mpv`, koji radi na praktično svakoj grafičkoj kartici i operativnom sistemu.

13 Zaključak

U radu je ispitana primena konvolucionih modela za superrezoluciju pojedinačne slike na uvećanje video-zapisa u realnom vremenu, gde se model primenjuje nezavisno nad svakim frejmom, pa se izbor modela ne zasniva na najboljem kvalitetu, već na kvalitetu koji se postiže unutar vremenskog budžeta jednog frejma. Šest arhitektura iz literature i predložena arhitektura FastEDSR trenirane su u velikom broju konfiguracija različite dubine i širine, a zatim primenjene u dva sistema za uvećanje video-zapisa.

Svi neuronski modeli nadmašuju klasične interpolacijske metode prema sve tri metrike. Najveće konfiguracije povećavaju PSNR za 2,9 do 3,4 dB u odnosu na bikubnu interpolaciju i približno dvostruko smanjuju LPIPS, dok su međusobne razlike arhitektura znatno manje od te prednosti.

Poređenje arhitektura pokazuje da efikasnost po broju parametara ne znači i efikasnost po vremenu izvršavanja. VDSR je najefikasniji po broju parametara, ali najsporiji, jer konvolucije izvršava nad s^2 puta više piksela. Destilacija i pažnja kod IMDN-a i RFDN-a smanjuju broj parametara, ali dodatne operacije produžavaju izvršavanje, pa ove arhitekture iznad približno 200 FPS naglo gube kvalitet. Najbolji odnos kvaliteta i brzine imaju jednostavne mreže, ABPN i FastEDSR, koje rade u LR prostoru, sastavljene samo od konvolucija, aktivacija i sabiranja i predviđaju rezidual nad interpolacijom uvećanim ulazom.

Pokazano je i da se više faktora uvećanja može objediniti u jednom višeskalnom modelu bez gubitka kvaliteta, kao i da treniranje sa JPEG degradacijom povećava PSNR na kompresovanim ulazima u proseku za 1,02 dB.

Izvršno okruženje i numerička preciznost utiču na brzinu podjednako kao i izbor arhitekture. FP16 je u proseku 2,13 puta brži od FP32. Na NVIDIA grafičkim karticama najbrže je direktno izvršavanje TensorRT engine-a. TensorRT provajder ONNX Runtime-a je 1,5 puta brži od DirectML-a, dok NCNN sa Vulkanom donosi prenosivost na druge operativne sisteme uz znatno manju brzinu.

Koraci van same inferencije čine značajan deo vremenskog budžeta. U osnovnoj varijanti plejera, za model FastEDSR 4/128 i video rezolucije 480p, bikubno skaliranje i prikaz na centralnom procesoru traju gotovo 5 ms, odnosno više od četvrtine ukupnih 17,68 ms. Zero-copy varijanta, sa hardverskim dekodiranjem i prikazom preko OpenGL-a, smanjuje ukupno vreme na 11,57 ms, čime isti model ulazi u budžet od 16,7 ms za 60 FPS. Dobitak je relativno najveći kod malih modela, kod kojih inferencija čini manji deo ukupnog vremena.

Konačno, modeli su primenjeni u dva sistema. Prvi je demonstracioni plejer u Python-u, koji celu obradu frejma obavlja na grafičkoj kartici, a drugi generator GLSL šejdera, koji FastEDSR prevodi u šejdere za postojeći plejer `mpv`, pa se superrezolucija može koristiti bez ikakvog izvršnog okruženja za neuronske mreže i sa punim mogućnostima video-plejera.

Literatura

[1] E. Meijering, "A chronology of interpolation: From ancient astronomy to modern signal and image processing," *Proceedings of the IEEE*, vol. 90, no. 3, pp. 319–342, 2002.

[2] R. G. Keys, "Cubic convolution interpolation for digital image processing," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 29, no. 6, pp. 1153–1160, 1981.

[3] C. E. Duchon, "Lanczos filtering in one and two dimensions," *Journal of Applied Meteorology*, vol. 18, no. 8, pp. 1016–1022, 1979.

[4] C. Dong, C. C. Loy, K. He, and X. Tang, "Image super-resolution using deep convolutional networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 2, pp. 295–307, 2016.

[5] J. Yang, J. Wright, T. S. Huang, and Y. Ma, "Image super-resolution via sparse representation," *IEEE Transactions on Image Processing*, vol. 19, no. 11, pp. 2861–2873, 2010.

[6] J. Kim, J. K. Lee, and K. M. Lee, "Accurate image super-resolution using very deep convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1646–1654, 2016.

[7] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

[8] B. Lim, S. Son, H. Kim, S. Nah, and K. M. Lee, "Enhanced deep residual networks for single image super-resolution," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 136–144, 2017.

[9] C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, J. Totz, Z. Wang, and W. Shi, "Photo-realistic single image super-resolution using a generative adversarial network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4681–4690, 2017.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[11] W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1874–1883, 2016.

[12] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.

[13] E. Agustsson and R. Timofte, "NTIRE 2017 challenge on single image super-resolution: Dataset and study," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 126–135, 2017.

[14] Z. Hui, X. Gao, Y. Yang, and X. Wang, "Lightweight image super-resolution with information multi-distillation network," in *Proceedings of the 27th ACM International Conference on Multimedia (ACM MM)*, pp. 2024–2032, 2019.

[15] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7132–7141, 2018.

[16] J. Liu, J. Tang, and G. Wu, "Residual feature distillation network for lightweight image super-resolution," in *Computer Vision – ECCV 2020 Workshops*, pp. 41–55, 2020.

[17] J. Liu, W. Zhang, Y. Tang, J. Tang, and G. Wu, "Residual feature aggregation network for image super-resolution," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2359–2368, 2020.

[18] Z. Du, J. Liu, J. Tang, and G. Wu, "Anchor-based plain net for mobile image super-resolution," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 2494–2502, 2021.
[19] Z. Wang, J. Chen, and S. C. H. Hoi, "Deep learning for image super-resolution: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 10, pp. 3365–3387, 2021.
[20] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
[21] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 586–595, 2018.
[22] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, pp. 1097–1105, 2012.
[23] PyTorch and NVIDIA, "Torch-TensorRT." `https://github.com/pytorch/TensorRT`.
[24] G. K. Wallace, "The JPEG still picture compression standard," *Communications of the ACM*, vol. 34, no. 4, pp. 30–44, 1991.
[25] K. Zhang, J. Liang, L. Van Gool, and R. Timofte, "Designing a practical degradation model for deep blind image super-resolution," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4791–4800, 2021.
[26] X. Wang, L. Xie, C. Dong, and Y. Shan, "Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pp. 1905–1914, 2021.
[27] PyTorch Developers, "PyTorch documentation." `https://docs.pytorch.org/docs/stable/`.
[28] ONNX Developers, "ONNX: Open neural network exchange." `https://onnx.ai/`.
[29] ONNX Runtime Developers, "ONNX Runtime." `https://onnxruntime.ai/`.
[30] ONNX Runtime Developers, "ONNX Runtime execution providers." `https://onnxruntime.ai/docs/execution-providers/`.
[31] Microsoft, "DirectML." `https://github.com/microsoft/DirectML`.
[32] NVIDIA, "CUDA C++ programming guide." `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.
[33] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cuDNN: Efficient primitives for deep learning," *arXiv preprint arXiv:1410.0759*, 2014.
[34] Intel, "OpenVINO toolkit." `https://github.com/openvinotoolkit/openvino`.
[35] AMD, "MIGraphX." `https://github.com/ROCm/AMDMIGraphX`.
[36] NVIDIA, "NVIDIA TensorRT documentation." `https://docs.nvidia.com/deeplearning/tensorrt/`.
[37] Tencent, "ncnn: High-performance neural network inference framework optimized for the mobile platform." `https://github.com/Tencent/ncnn`.
[38] The Khronos Group, "Vulkan specification." `https://registry.khronos.org/vulkan/`.
[39] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.
[40] PyAV Developers, "PyAV: Pythonic bindings for FFmpeg libraries." `https://github.com/PyAV-Org/PyAV`.
[41] M. Geier, "python-sounddevice: Play and record sound with Python." `https://github.com/spatialaudio/python-sounddevice`.
[42] NVIDIA, "NVIDIA Video Codec SDK." `https://developer.nvidia.com/video-codec-sdk`.

[43] NVIDIA, "PyNvVideoCodec." https://developer.nvidia.com/pynvvideocodec.
[44] DLPack Developers, "DLPack: Open in-memory tensor structure." https://github.com/dmlc/dlpack.
[45] The Khronos Group, "OpenGL registry." https://registry.khronos.org/OpenGL/.
[46] GLFW Developers, "GLFW." https://www.glfw.org/.
[47] NVIDIA, "CUDA Python." https://github.com/NVIDIA/cuda-python.
[48] M. C. Fletcher, "PyOpenGL." https://pyopengl.sourceforge.net/.
[49] mpv Developers, "mpv: A free, open source, and cross-platform media player." https://mpv.io/.
[50] J. Kessenich, D. Baldwin, and R. Rost, "The OpenGL shading language, version 4.60." The Khronos Group, https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf.
[51] mpv Developers, "mpv manual: GLSL shaders." https://mpv.io/manual/master/#glsl-shaders.